CHIME: A Case for Efficient Long-Context Attention-FC Disaggregated Inference with DIMM-PIM

Qingyuan Liu*, Liyan Chen*, Haocheng Wang, Yanning Yang, Dong Du,
Zhigang Mao, Naifeng Jing, Yubin Xia, Haibo Chen
Shanghai Jiao Tong University, Shanghai, China

***Abstract*—Attention-FC Disaggregated (AFD) LLM inference systems offload memory-bound Attention operations to memory-rich accelerators (e.g., CPUs, HBM-PIM) while retaining compute-bound Fully-Connected (FC) operations on GPUs. In this paper, we first design a Disaggregated Roofline Model (DRM) to characterize AFD performance, revealing that *system throughput is constrained by the accelerator's limiting factor: either memory bandwidth or capacity*. We observe that prior AFD systems often overlook these constraints and fail to balance them, leading to resource underutilization or constrained throughput. Therefore, we propose CHIME, the first AFD system integrating *DIMM-PIM*, which is a case of the new accelerator that strikes the balance with scalable capacity and bandwidth. To address the synchronization challenges inherent to the distributed cooperating DRAM chips in DIMM-PIM, CHIME employs *bubble-free pipelining* and *hybrid-grained re-layout* for efficient attention computation. Furthermore, it maximizes cross-device resource utilization via *rankset-granular communication-computation overlapping* and *alignment-predicting scheduling*. Evaluations show CHIME achieves up to 5.15× speedup over state-of-the-art HBM-PIM solutions.**

I. Introduction

"A plant's growth is limited by the nutrient in shortest supply, regardless of how abundant other nutrients may be."

—*Liebig's Law* [75], Justus von Liebig (1803-1873).

The demand for Large Language Models (LLMs) to process and generate longer sequences is rapidly increasing, driven by applications like long reasoning and complex code generation [16], [21], [29], [65], [81], [82]. In these long-context scenarios, the decoding process becomes a significant performance bottleneck. Moreover, the increasingly longer KV cache imposes growing memory capacity and bandwidth demands on the inference system.

To tackle this memory challenge, Attention-FC Disaggregation (AFD) has emerged as a promising solution [27], [28], [30], [61], [71], [72], [83]. AFD systems partition the LLM inference workload: the memory-intensive Attention operations and KV cache are offloaded to a specialized memory-rich device (hereafter referred to as the *"accelerator"*), while the compute-intensive Fully-Connected (FC) operations remain on the GPU or NPU. For example, leveraging HBM-based Processing-in-Memory (HBM-PIM) can dramatically reduce the latency of Attention [27], [28], [61] with significantly increased bandwidth. In conclusion, AFD systems show the potential for higher throughput.

However, the promise of AFD architecture is not always straightforward to realize. Our investigation reveals a critical yet counter-intuitive phenomenon: **simply enhancing the accelerator does not always guarantee better system performance.** In an evaluation on GPT-175B with OpenR1 [5], we found that equipping DGX-A100's HBMs with PIM from 1× to 16× higher bandwidth only leads to <1% throughput improvement. This puzzling outcome cannot be explained by existing performance models, such as the classic roofline model [76], which analyzes devices in isolation and fails to capture the complex interplay between disaggregated components in an AFD system.

To address this, we design the **Disaggregated Roofline Model (DRM)**(§III), the first general performance model to analyze how AFD systems perform in various scenarios. DRM holistically characterizes the performance of both the accelerator and the GPU, crucially modeling their interdependencies. Using DRM, we conclude a principle guiding the design for AFD systems: the system's throughput is limited by the scarcest resource, be it the accelerator's memory bandwidth or capacity. Crucially, improvements to any non-bottleneck resource will result in diminishing or even negligible returns. It explains the phenomenon described in the previous paragraph, and motivates the design of a new accelerator that achieves a more balanced tradeoff among capacity, bandwidth, and scalability.

In this paper, we show that PIM integrated with *DIMM*, i.e., equipping host memory modules with bank-level processing units, constitutes a case of a new accelerator that strikes this balance. Therefore, we propose AFD systems that leverage *DIMM-PIM*. Compared to prior HBM/GDDR-PIM solutions [22], [27], [28], [61], an AFD system with DIMM-PIM gains three key advantages. First, it directly alleviates the capacity bottleneck of HBM-PIM while providing significantly higher bandwidth than standard host memory, enabling higher overall throughput. Second, it inherits the superior scalability and configurability of the standard DIMM interface, making the system adaptable and future-proof for diverse memory demands. Third, as analyzed in §III-C, it offers higher economic efficiency, which could be cheaper

*Co-first author.

979-8-3315-5065-3/26/$31.00 ©2026 IEEE
DOI 10.1109/ISCA66397.2026.00056

than HBM/GDDR-PIM and suffer less memory wastage.

However, effectively leveraging DIMM-PIM in AFD systems is non-trivial. A naive integration of DIMM-PIM introduces additional synchronization overheads both within the PIM hardware and across the inference system, including transfer, layout, and progress synchronization, which can severely stall inference. Specifically:

First, compared with prior HBM-PIM/GDDR-PIM architectures [22], [27], [28], [61], the DIMM-PIM architecture features *multiple distributed, cooperating DRAM chips*, which aggravate two synchronization overheads: (1) *Data synchronization:* attention computation distributed across multiple DRAM chips necessitates data synchronization via cross-chip transfers, e.g., accumulating partial outputs across chips for "softmax". (2) *Layout synchronization:* the DIMM-based data storage layout may not match the layout required by PIM computation: for instance, a single element may be striped across several chips, while PIM computation requires each element resides in a single chip. Aligning the layout thus becomes a prerequisite for attention execution. These overheads introduce substantial stalls in PIM execution, preventing DIMM-PIM from achieving its theoretical performance.

Second, the disaggregation of the inference process between the GPU and DIMM-PIM also introduces two types of synchronization overhead: (1) *Data synchronization:* PIM needs to fetch the Q,K,V tensors from the GPU and return the attention outputs to the GPU. These transfers via PCIe can stall PIM execution. (2) *Progress synchronization:* when parallel operations on the PIM and GPU have imbalanced completion times, the device that finishes earlier is forced to wait, creating idle "bubbles". This issue is compounded by the fact that the execution latencies of parallelizable tasks can vary, making workload alignment more challenging.

To address these challenges, we propose CHIME, the first AFD system that integrates a novel DIMM-PIM design as the accelerator, reducing synchronization overheads from the hardware internals to the overall system. First, we design CHIME-PIM (§V), a DIMM-PIM hardware for efficient attention computation. It eliminates the overheads of data and layout synchronization with two key techniques respectively: *bubble-free pipelining* and *hybrid-grained relayout*. Specifically, the pipeline overlaps data transfer with concurrent bank PU execution to hide the transfer overheads, which further enables bubble-free with specific head mapping according to a quantitative analysis. The hybrid-grained relayout performs data layout transformation at element level (coarse-grained) and bit level (fine-grained) to address the layout mismatch with minimum latency.

Second, we design CHIME-sys (§VI), a DIMM-PIM integrated inference system that reduces the overheads of data and progress synchronization respectively with *rankset-granular communication computation overlapping* and *alignment-predicting scheduling*. Rankset-granular communication computation overlapping exploits the finest granularity of independent communication and computation, i.e., *the rankset*, which consists of one rank from each channel. This design achieves parallel communication and computation across ranksets to support asynchronous data transfer, effectively hiding the communication overhead between the GPU and CHIME-PIM with computation. Alignment-predicting scheduling selects requests to form sub-batches with the ability of modeling the performance of operations on both devices, selecting requests accordingly to form sub-batches whose predicted latencies on the two devices are aligned. This maximizes the parallel execution on the two devices with minimum idle bubbles.

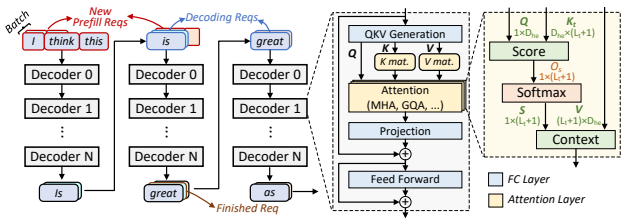


Fig. 1. **LLM inference process.**

Comprehensive evaluations on three LLM models and three real-world traces [2], [5], [6], [11] show that CHIME achieves up to 5.15× speedup over state-of-the-art HBM-PIM solutions [28], [61] with significantly improved batch sizes.

II. The Rise of AF-Disaggregated Systems

A. LLM Inference Process

Fig. 1 illustrates the LLM inference process [9], [19], [21], [33], which includes two stages, i.e., prefilling and decoding. The LLM model constitutes a sequence of layers, each comprising two major kinds of operations: (1) attention operations [73] and (2) fully-connected (FC) operations, which encompass QKV Gen, projection, Feed-forward Network, etc. MHA and GQA are leading attention operators, with MHA computing each head independently and GQA grouping query heads for parallel processing. The computations of attention consist of independent heads, each following score ($Q \times K_t$), softmax, and context computations ($S \times V$).

B. Attention-FC Disaggregated Inference System

Attention and FC operations typically have different characteristics. Specifically, benefiting from batching [8], [41], [78], FC operations are compute-intensive and demand compute resources for acceleration. On the contrary, decoding attention is typically bandwidth-intensive [61] and does not benefit from batching, since the KV cache is distinct and specific for each request. The latency of attention is primarily determined by memory bandwidth for loading the KV cache. Considering the different characteristics of the two operations, some current works, i.e., Attention-FC disaggregated inference systems ("AFD systems" in short), disaggregate the attention and FC operations on different hardware platforms. Specifically, they batch FC operations on the compute-rich GPU/NPU, while offloading the KV cache and the decoding attention to a platform (called *accelerators*) better suited for its memory-intensive nature.

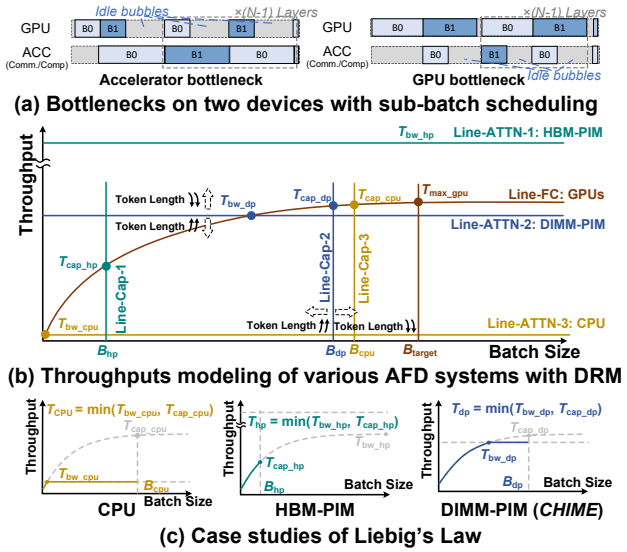


Fig. 2. **CHIME's Disaggregated Roofline Model and case studies.** *"DIMM-PIM" denotes CHIME's proposal, leveraging DIMM-based host memory with bank-level PUs as the accelerator. "Throughput" denotes tokens per second.*

To improve the overall utilization, current AFD systems typically apply *sub-batch scheduling* [28], [30], [83], making two (or even more) interleaved sub-batches to run in parallel across the two devices, as shown in Fig. 2-a. During inference, the execution of an individual batch cannot be parallelized on two devices, e.g., GPU has to wait for the completion of attention before starting FC. Sub-batch scheduling enables attention and FC of different batches to run concurrently on the two devices, which could improve the throughput and resource utilization of AFD systems.

C. Processing-In-Memory (PIM)

PIM is a promising solution to offer high aggregated bandwidth by integrating processing units (PUs) in memory devices, such as HBM [26], [38], [39], [46], [49], GDDR [36], [42], [43], [47], [48], and DIMM [18], [34], [35], [44]. For example, with two DIMMs (e.g., two ranks of 16 banks per DIMM) in one channel, placing PUs near ranks (**rank PU**) can achieve about 4× the bandwidth of host CPU, while integrating PUs near banks (**bank PU**) can improve bandwidth by more than 30×. In this case, some state-of-the-art AFD systems are built upon PIM devices, such as HBM-PIM [27], [28], [61] or GDDR-PIM [22], delivering substantial performance gains over GPUs.

III. Characterizing AFD Systems with DRM

In this section, we first introduce **Disaggregated Roofline Model (DRM)**, a unique analytical model specifically designed for AFD systems, characterizing the performance across diverse scenarios. Based on the model, we analyze the memory bottlenecks for current AFD systems whose attention is offloaded to various memory-enhanced devices, including HBM-PIM, which boosts bandwidth, and CPUs (with host memory), which expand capacity. According to the analysis, we conclude how the "Liebig's Law" manifests for AFD systems, which informs the design of accelerators with effective memory enhancement to achieve high throughput.

A. DRM Modeling and Bottleneck Analysis

Characterizing the throughput of AFD systems requires comprehensively considering two factors: the throughput on the GPU side for executing FC operations, and the throughput on the accelerator side for executing attention operations. We first identify that when the throughputs on the two devices are not equal, the throughput of the inference system is only affected by the device that becomes the bottleneck. For example in the right of Fig. 2-a, the bottleneck is the GPU, and the accelerator suffers idle bubbles consequently. In this case, increasing the throughput on the accelerator side will only result in more idle bubbles, having no effect on improving the overall throughput. The left of Fig. 2-a shows an opposite example. In conclusion:

> **Implication I** *The overall inference throughput of AFD systems is determined by the lower throughputs of the two devices that separately execute the FC and attention.*

Construction of DRM. Fig. 2-b presents a conceptual illustration of the DRM, which uniformly characterizes the performance of two disaggregated devices by modeling the relationship between batch size and token throughput (tokens per second). Token throughput is derived from two key metrics: (1) floating point operations per second, which is dictated by the batch-size-dependent arithmetic intensity based on traditional roofline model. (2) floating point operations, which scales with the batch size. Line-FC shows the GPU token throughput for executing the FC, which increases with the growth of batch size until the GPU is fully utilized, i.e., when the batch size reaches B_{target}, the GPU achieves peak throughput $T_{\text{max\_gpu}}$. Line-ATTN-1, 2, 3 show the token throughputs of different accelerators for executing the attention respectively. Since the arithmetic intensity of attention remains constant and low, these throughputs are typically positively correlated with accelerator memory bandwidth, regardless of batch size. Moreover, The performance curves in Fig. 2-b can be not only theoretical, but also profiled.

Bandwidth constraints. According to Implication I, the throughput of an AFD system is determined by the lower value between the throughput curves of the GPU and the accelerator. There can be two types of situations: First, the throughput curves of the GPU and the accelerator intersect, e.g., when the accelerator is the CPU. The GPU throughput may initially be lower than that of the CPU when the batch size is very low, and the throughput of the AFD system is determined by Line-FC. As the batch size increases, the GPU throughput quickly rises to $T_{\text{bw\_cpu}}$, after which the throughput of the AFD system is determined by Line-ATTN-3, which is constrained by the CPU memory bandwidth. Second, if the throughput curves of the GPU and the accelerator do

not intersect, the throughput of the AFD system is always determined by the lower curve. E.g., if the accelerator is HBM-PIM in Fig. 2-b, the throughput is always limited by the GPU side (Line-FC).

Implication II *The bandwidth of the accelerator limits the throughput of AFD systems when the attention becomes the bottleneck.*

Capacity constraint. Another factor is the memory capacity on the accelerator side to store the KV cache. Insufficient memory capacity restricts the maximum number of requests that can be accommodated, preventing the GPU side from fully utilizing GPU resources. In Fig. 2-b, Line-Cap-1, 2, 3 show the maximum batch sizes that can be accommodated by various accelerators, i.e., B_{hp}, B_{dp}, B_{cpu} for HBM-PIM, DIMM-PIM and CPU respectively. The GPU throughput for executing the FC is thus limited by these maximum batch sizes. For example, when the accelerator is the HBM-PIM, the maximum GPU throughput on Line-FC is limited below $T_{\text{cap\_hp}}$, since the batch size of this AFD system is at most B_{hp}. When the bottleneck is on the GPU side, e.g., HBM-PIM as the accelerator, the AFD system's throughput is constrained by the limited batch size.

Implication III *The capacity that stores the KV cache limits the overall throughput of AFD systems when the FC becomes the bottleneck.*

Dynamic workloads and shifting rooflines. As DRM is an analytical model for design-time architectural analysis, its purpose is not to predict the precise latency of a given request [32], [70]. Nevertheless, DRM could also illustrate how the performance rooflines shift in response to varying runtime workload characteristics. For example, with longer context, the maximum batch size that can be accommodated within the same capacity and the throughput of processing a larger KV cache with the same bandwidth decreases, so it causes the capacity lines (Line-Cap-1, 2, 3 in Fig. 2-b) to shift leftward and the Line-ATTN-1, 2, 3 to shift downward, while for the GPU, Line-FC remains almost unchanged. Consequently, it illustrates how longer contexts exacerbate memory bottlenecks along both the bandwidth and capacity.

Characterizing communication overhead. The methodology of DRM is also capable of characterizing the communication overhead between GPUs and attention accelerators, considering it as a part of attention overhead. Within the DRM, the decrease of communication bandwidth (e.g., utilizing PCIe 3.0 instead of PCIe 4.0) results in a downward shift of the attention performance curve. According to our detailed analysis in §VI-A, the communication overhead in AFD systems is constant, and is optimized by our design.

The "Liebig's Law" for AFD systems. Considering Implication II and III, for a given GPU configuration that executes FC, the accelerator's memory bandwidth and capacity respectively limit the throughput when the attention or FC becomes the bottleneck. Further, according to Implication I, this indicates that the overall throughput is determined by the memory capacity or bandwidth whose throughput limit is lower, which we refer to as the weaker point of memory. We conclude that the accelerator's memory impact on overall throughput follows Liebig's Law, that is:

Liebig's Law: *The throughput of an AFD system is constrained by the weaker point in terms of either memory capacity or bandwidth.*

Although Liebig's Law is a general principle applicable to any system, the analysis methodology with DRM first reveals how the Law manifests for AFD systems.

Case study. Fig. 2-c extracts concrete examples from Fig. 2-b as case studies. The weaker point of the "CPU" is the limited bandwidth of the host memory, so the throughput of the CPU AFD system is constrained by $T_{\text{bw\_cpu}}$ even if it has sufficient memory capacity to accommodate larger batch sizes. The weaker point of the "HBM-PIM" is the limited HBM capacity, so the throughput is constrained by $T_{\text{cap\_hp}}$ even if it can execute attention extremely quickly.

Quantitative analysis on Liebig's Law. To illustrate Liebig's Law quantitatively, we extend the illustrative figure of Fig. 2 to Fig. 3. Fig. 3 quantifies the throughput limitation assuming 8×A100 of DGX-A100 are used for computing FC, while using accelerators for attention with various memory configurations. It clearly demonstrates Liebig's Law, i.e., better throughput can only be achieved when both capacity and bandwidth are scaled simultaneously. If only one of them is scaled, for example, fixing the capacity and scaling the bandwidth, the throughput will only improve with increasing bandwidth until the bandwidth reaches a certain point. Beyond that point, higher bandwidth yields no further improvement, as capacity becomes the bottleneck.

B. Limitations on Prior Works

Existing works have made considerable efforts to enhance memory for improving inference throughput. However, they mainly focus on addressing one dimension of the memory bottleneck, either capacity or bandwidth. Their neglect of Liebig's Law renders the resulting performance gains potentially inefficient or insufficient. To illustrate, we plot the hardware configurations of various hardware types under scaling scenarios in Fig. 3: including HBM2e, GDDR, two types of DDR4 memory and their PIM variants. Some listed systems [22] in Fig. 3 are not specifically designed for AFD, and we mainly consider their hardware for executing attention. We can conclude that prior works suffer from the "Liebig's Law" in the following aspects:

Insufficient memory enhancement. First, without adequate scaling, these memory enhancement techniques could be insufficient to prevent memory from becoming a throughput bottleneck. This can be illustrated by examining the hardware parameters claimed in their respective papers in

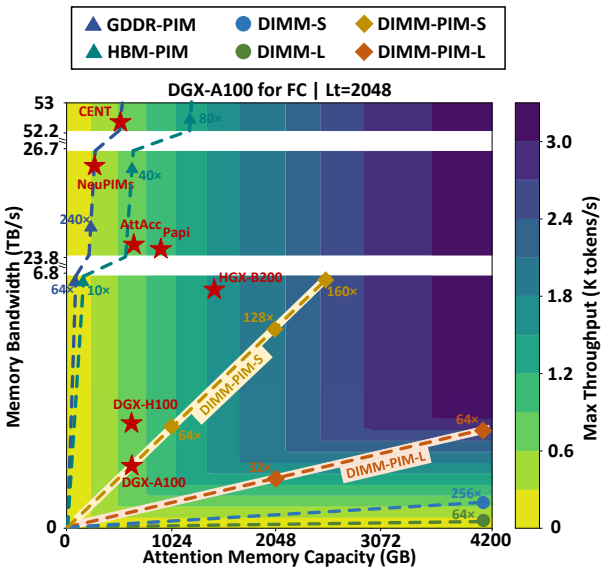
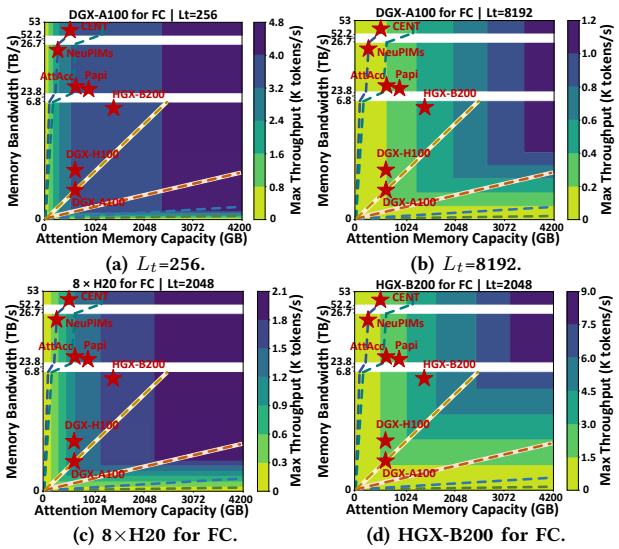


Fig. 3. **Contour plot of the throughput with memory bottlenecks.** *"L_t=2048" above denotes that each request has a context length of 2048 tokens. Memory configurations of "PAPI", "NeuPIMs", "AttAcc" and "CENT" are derived from the papers [22], [27], [28], [61]. "DGX-A100", "DGX-H100" and "HGX-B200" [1], [57], [58] represents using GPUs for attention. DIMM-S, DIMM-L denote two different types of DDR4 memory. Every point represents the total memory capacity and bandwidth under the corresponding number of memory chips, for example, the point "64×" on the "DIMM-L" line (green line) represents the memory capacity and bandwidth corresponding to 64× DIMM-L memory chips. We assume that the PIM performance of each memory device can reach the ideal case, i.e., the equivalent memory bandwidth is scaled by 16×. The performance is simulated with AttAcc [61] simulator with GPT-175B model.*



(a) L_t=256. (b) L_t=8192.

(c) 8×H20 for FC. (d) HGX-B200 for FC.

Fig. 4. **Contour plot of the throughput for four scenarios.** *This figure shares the legend with Fig. 3. Compared to the scenario in Fig. 3, (a) and (b) vary the context lengths, while (c) and (d) adjust the number of GPUs used for FC computation.*

conjunction with Fig. 3. Specifically, works based on HBM-PIM could suffer from insufficient memory capacity [27], [28], [61], while others suffer from insufficient memory bandwidth of DIMMs [30], [52].

Inefficient memory enhancement. Further, current works suffer from excessive memory resources. For example, although current GDDR/HBM-PIM based methods [22], [27], [28], [61] are limited by capacity, their bandwidth could be orders of magnitude larger than the requirement, which provides almost no benefit to throughput. Moreover, even if scaling them could potentially achieve high throughput, such scaling could simultaneously lead to substantial bandwidth waste, as illustrated in Fig. 3.

C. Proposal of Choosing DIMM for PIM Intergration

Based on our analysis, we find that integrating PIM with *DIMM*s (i.e., *DIMM-PIM*) is a promising accelerator for attention. Specifically, *DIMM-PIM* augments DIMM-based host memory with bank-level PIM units, combining the strengths of PIM and *DIMM*s. Therefore, we propose AFD systems that integrate *DIMM-PIM*, which offer the following benefits:

Better performance with more balanced memory configuration. Typically, DIMM-PIM can alleviate the capacity bottleneck of HBM-PIM and the bandwidth bottleneck of CPU offloading. As illustrated in Fig. 2-b, c, this means that DIMM-PIM potentially improves the overall throughput with a more balanced memory configuration. As shown in Fig. 3, since the DGX-A100 is equipped with 2TB of DIMM memory (DDR4) and 640GB of HBM memory, equipping the DIMMs with PIMs already significantly outperforms equipping the HBMs with PIMs.

Scalable and configurable. DIMM-PIM offers scalability via the standard DIMM form factor, enabling plug-and-play capacity/bandwidth scaling on compatible platforms, and can be further scaled using interconnects such as CXL. This allows DIMM-PIM to not only achieve better performance but also potentially adapt to a broader range of scenarios.

Economic efficiency. Beyond performance considerations, DIMM-PIM also offers a significant cost advantage over GDDR- and HBM-based alternatives. First, the per-GB cost of HBM2e can be over 6× higher than DDR4 [3], [4], [7], which represents a substantial difference, regardless of whether PIM is integrated. Second, Fig. 3 shows that, the more balanced performance of DIMM-PIM avoids the significant resource over-provisioning inherent in HBM/GDDR-PIM solutions, enabling more cost-effective performance scaling.

D. Discussion: Bottleneck Shifting across Various Scenarios.

In practice, AFD systems may adopt various deployment configurations, which could shift the memory bottleneck. The deployment configurations include GPU model and quantity, context length, LLM parameter size and architecture, etc. An important contribution of our DRM-based methodology is its ability to analyze, for a given deployment configuration, how the memory configuration of each accelerator affects overall throughput. In this section, we further discuss how

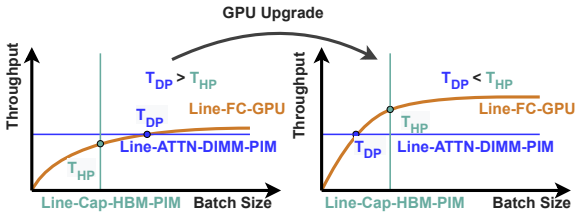


Fig. 5. **DRM with GPU upgrade.** *T_{DP} and T_{HP} denote the throughput of DIMM-PIM under its memory bandwidth constraint, and the throughput of HBM-PIM under its memory capacity constraint, respectively.*

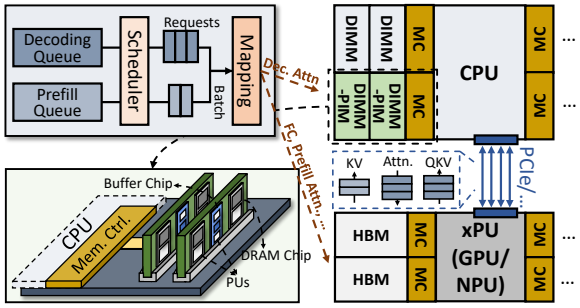


Fig. 6. **CHIME overview.**

different deployment configurations could affect the memory bottleneck.

Different context lengths. Fig. 3 and Fig. 4-a, b show how different context lengths affect the performance. First, we observe that across various context lengths, the more balanced configuration of DIMM-PIM consistently holds a performance and efficiency advantage, effectively improving throughput while reducing resource over-provisioning. It is because longer contexts exacerbate memory capacity and bandwidth bottlenecks simultaneously.

Second, we observe that with shorter contexts, the performances of various baselines become more comparable, because the marginal benefit of adding resources diminishes when the memory bottleneck is alleviated. We validate this analysis in §VII-B. The impact of context length can also be analyzed in conjunction with Fig. 2-b.

Influence of other configurations. Our DRM method can also analyze the impact of other configuration changes on accelerator selection. For example, with more advanced GPUs for executing FC, as shown in Fig. 5, the DRM reveals that this shifts the curve of FC throughput upward, enhancing HBM-PIM throughput while leaving DIMM-PIM throughput unchanged, thereby closing the performance gap of HBM-PIM relative to DIMM-PIM and even turning it into an advantage. Fig. 3 and Fig. 4-c, d show examples of this: 32× DIMM-PIM-L outperforms AttAcc [61] with A100 (Fig. 3), but underperforms AttAcc when paired with B200 (Fig. 4-d). Other scenarios, such as varying the number of GPUs used for FC, can also be analyzed following a similar methodology.

IV. Overview of CHIME

A. Design Overview

To support the proposal of integrating DIMM-PIM, as shown in Fig. 6, we propose CHIME that consists of two parts: (1) CHIME-PIM, our specifically designed DIMM-PIM hardware for efficient attention execution; and (2) CHIME-sys, a scalable AFD system which coordinates the computation on GPU and CHIME-PIM and aims at maximizing the inference throughput.

Inference workflow of CHIME. CHIME-sys inherits the sub-batch scheduling technique [28], [30], selecting requests to form sub-batches for each iteration. Initially, for each sub-batch, QKV Generation of all requests is batched and executed on the GPU. Then, prefilling and decoding attention are processed concurrently on the GPU and CHIME-PIM respectively. CHIME aggregates attention outputs from all requests in the batch, and then performs batched execution of subsequent operations (projection, Feed-forward, etc). PCIe interconnects the GPU and CHIME-PIM.

Addressing the challenges of integrating DIMM-PIM. As outlined in §I, integrating DIMM-PIM introduces architectural disaggregation and, consequently, synchronization overheads: within a DIMM, distributed chips incur both data-synchronization and layout-synchronization costs, while the disaggregation between DIMM-PIM and the GPU adds the overhead of cross-device data synchronization and progress synchronization. To address the former, CHIME-PIM employs bubble-free pipelining (§V-A) and hybrid-grained re-layout (§V-B); to address the latter, CHIME-sys employs rankset-granular communication-computation overlapping (§VI-A) and alignment-predicting scheduling (§VI-B).

V. Attention Acceleration on CHIME-PIM

This section describes CHIME-PIM, a novel DIMM-PIM hardware design for efficient attention computation.

Hardware components. As shown in Fig. 7-a, CHIME-PIM integrates co-operated processing units (PUs) across both bank and rank levels. The bank PUs are integrated near DRAM banks with DRAM process, fetching KV cache from banks and performing score and context computation, while a shared buffer is leveraged to broadcast input vectors to all bank PUs. The bank PUs in all DRAM chips perform multiplication-and-accumulation (MAC) concurrently and generate outputs to the result buffer. Softmax unit, adder unit, and re-layout unit are integrated on the buffer chip with logic process as a part of the rank PU, which can execute in an asynchronous manner with bank PUs.

The CHIME-PIM workflow avoids modifications to host CPU memory controllers, which works as follows: first, CPU offloads PIM requests to the rank PU via normal writes, which are further decoded to PIM commands. Then, a dedicated PIM controller issues PIM commands on standard DDR interface to DRAM chips to trigger corresponding operations. PIM commands include the ones for metadata setup and data

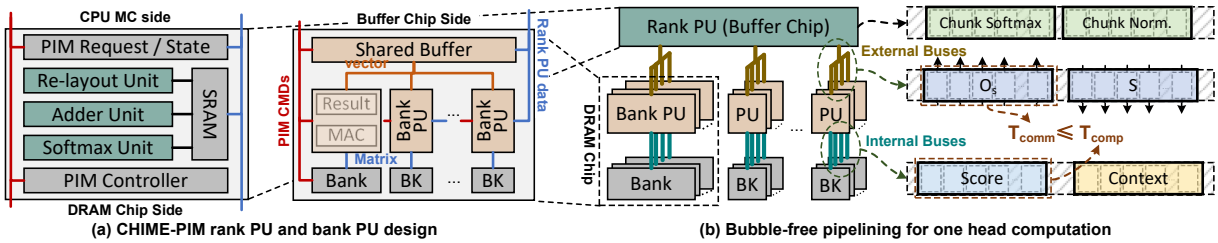


Fig. 7. **CHIME-PIM design with bubble-free pipelining.**

movement, and some key commands are listed as follows. PIM_WR_R writes necessary information to registers, such as configurations of computing paradigm (adder tree or accumulator), self-incrementing index mechanisms of buffers, and so on. PIM_LD_SB loads data from a certain bank to the shared buffer, while PIM_WR_SB writes data from rank PU to the shared buffer. PIM_MAC executes MAC operations in all banks while reading necessary data from DRAM cells and shared buffer. PIM_RD_RB loads data from result buffers of all banks to rank PU.

A. Bubble-free pipelining

To eliminate data-synchronization overhead in attention computation, the key is to leverage DIMM-PIM's unique architecture to design a pipelined attention execution that hides communication behind computation. Furthermore, through quantitative analysis and a tailored head-mapping method, we ensure that the communication cost is completely hidden.

Orchestrating the pipeline with decoupled memory buses and kernel fusion. We orchestrate a pipelined execution for attention kernels in PIM that aggressively overlaps computation and communication, grounded in two architectural observations. First, the internal memory buses (servicing bank PUs accessing banks) and external memory buses (handling rank PUs accessing bank PUs or shared buffer) can be decoupled [13], enabling simultaneous execution of compute kernels on bank PUs and data transfer from/to rank PU, as shown in Fig. 7-b. Second, inspired by FlashAttention's kernel fusion technique [15] (where attention is computed in chunked tiles), we orchestrate fine-grained kernel pipelining to maximize concurrency while constraining the intermediate head footprint on rank PUs.

Concretely, during the scoring phase, when each bank PU computes a token-specific output O_s and stages it in its local result buffer, the rank PU immediately fetches it via otherwise-idle external buses. Our attention kernel operates at the granularity of chunked tiles, where each chunk consists of data generated in parallel by a single head across (possibly) multiple bank PUs. After accumulation in adder unit, the softmax unit applies per-chunk softmax operations. In this way, score and softmax computations are pipelined between bank PUs and rank PU in a chunk-based manner. Following the processing of all tokens, a final cross-chunk normalization pass produces the globally-correct softmax output S, which is also performed in a streaming, chunk-wise manner. It allows the computation of finalized S elements, the write-back communication of S to DRAM chips, and subsequent context computations to be pipelined.

Enabling bubble-free with quantitative analysis and specific head mapping. Due to the nature of DIMM that multiple co-operated DRAM chips form a rank, KV matrix (cache) and related computation would be distributed across these DRAM chips. It multiplies the amount of data transfer between buffer chip and DRAM chips, which we refer to as the cross-chip data transfer. The cross-chip transfer overhead, even if being overlapped with the execution of the bank PU, can still lead to bubbles in pipelining execution and become the bottleneck due to the limited bandwidth of rank PU. To address these challenges, we first quantize the cost of cross-chip data transfer considering the configurations of LLM models, hardware parameters, and head mapping methods. Then, we enable bubble-free pipelining with specific head mapping methods. The analysis takes score computation of one head as an example, while context computation can follow the similar analysis.

The overheads for transferring score outputs O_s can be denoted as T_{comm}, following $T_{comm} = N_{comm}/B_{comm}$, where N_{comm} is the amount of cross-chip data transfer, and B_{comm} is the related bandwidth. Specifically:

$$N_{comm} = L_t \times N_{gqa} \times N_{hc}, B_{comm} = (B_{rk} \times N_{hc})/N_{chips}$$

where L_t is token length, N_{gqa} is GQA group size, N_{hc} is the number of chips allocated for the head, B_{rk} is rank bandwidth, and N_{chips} is total DRAM chips in the rank. Thus, the communication time is:

$$T_{comm} = (L_t \times N_{gqa} \times N_{chips})/B_{rk}$$

which implies that with DIMM's distributed chips, the data transfer could incur $N_{chips}\times$ additional overhead, while GQA size further exacerbates it.

With the pipelined execution, the key of achieving bubble-free overlapping is $T_{comm} \leq T_{comp}$, i.e., the transfer can be fully overlapped by the bank PU computation. T_{comp} can be calculated by $T_{comp} = N_{comp}/B_{comp}$, where N_{comp} is the amount of bank PU data (K Cache) fetching, and B_{comp} is the aggregated bank PU bandwidth. Specifically:

$$N_{comp} = L_t \times E_h \times \lceil N_{gqa}/N_{cmr} \rceil, B_{comp} = B_{bk} \times N_{bk} \times N_{hc}$$

where E_h is head embedding size, B_{bk} is bank PU bandwidth, N_{bk} is bank number, and N_{cmr} denotes the compute-memory

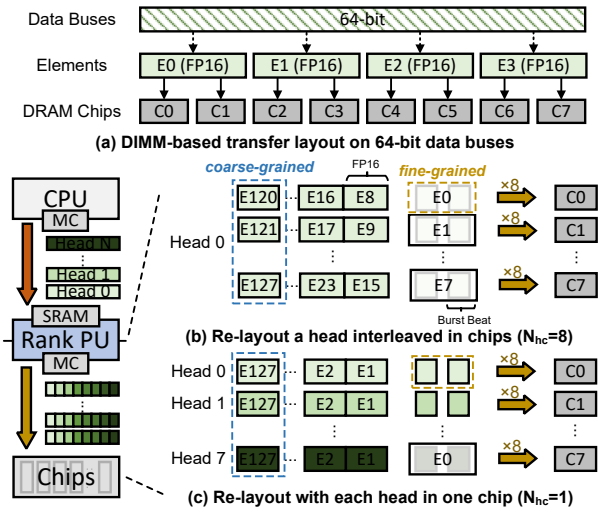


Fig. 8. **Hybrid re-layout with 8 ×8 chips as an example.** *E0-E127: elements 0-127 in a head. C0-C7: DRAM Chip 0-7.*

ratio of arithmetic units in the bank PU. Generally, N_{cmr}=1 can maximize bandwidth utilization for MHA computation, while N_{cmr}=8 for GQA-8 computation [61]. Thus, the computation time is

$$T_{comp} = (L_t \times E_h \times \lceil N_{gqa}/N_{cmr} \rceil)/(B_{bk} \times N_{bk} \times N_{hc})$$

Considering both T_{comm} and T_{comp}, the key for ensuring bubble-free is $T_{comm} \leq T_{comp}$:

$$\frac{L_t \times N_{gqa} \times N_{chips}}{B_{rk}} \leq \frac{L_t \times E_h \times \lceil N_{gqa}/N_{cmr} \rceil}{B_{bk} \times N_{bk} \times N_{hc}}$$

To satisfy the inequation, we identify given specific model and hardware configurations, the head mapping method (N_{hc}) is the only tunable variable. This requires our head mapping to satisfy the following condition:

$$N_{hc} \leq \frac{E_h \times B_{rk} \times \lceil N_{gqa}/N_{cmr} \rceil}{B_{bk} \times N_{bk} \times N_{gqa} \times N_{chips}} \quad (1)$$

For example, referring configurations in Table I and II, our system requires $N_{hc} \leq 8$ for MHA ($N_{gqa} = 1$) and $N_{hc} \leq 1$ for GQA-8 ($N_{gqa} = 8$). In this paper, we apply $N_{hc} = 8$ for MHA and $N_{hc} = 1$ for GQA-8, since mapping a head to more chips potentially simplifies rank-level load balance because heads of models can be mapped to more ranks.

B. Hybrid-grained Re-layout

Besides specific head-mapping layouts, DIMMs with distributed DRAM chips also impose constraints on element layout, as shown in Fig. 8-a. For contiguous transfers on the data buses, a single element may span multiple DRAM chips (e.g., an FP16 element across two ×8 chips), which fundamentally prevents PIM execution. Prior CPU-assisted re-layout schemes [10], [18], [24], [37], [45] require repeated memory accesses, incurring non-negligible overhead (§VII-C). To address this, we propose *hybrid-grained relayout*, which leverages the rank PU's re-layout unit to perform in-flight data transformation during communication.

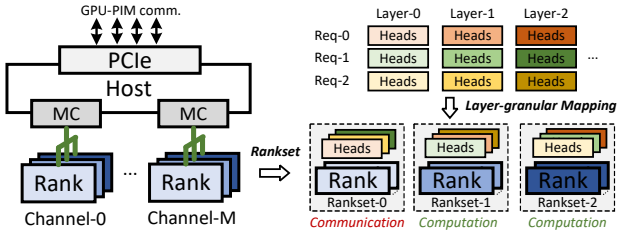


Fig. 9. **Hiding data transfer overhead with rankset-granular communication computation overlapping.**

The left of Fig. 8 shows the re-layout process with data offloading as an example. The QKV vectors are first buffered in the rank PU's on-chip SRAM rather than being written directly to DRAM banks. After re-layout, the vectors are then stored in the DRAM chips. The re-layout unit addresses mismatches between element mapping and head mapping via fine-grained and coarse-grained re-layout, respectively: First, for fine-grained re-layout, the re-layout unit ensures that each element resides on a single chip. As shown in Fig. 8, the bits of a single element are arranged in contiguous burst beats to be stored in one chip (brown dashed block). For instance, the 16 bits of element 0 (E0) are transferred in two contiguous burst beats (within a single DDR burst), and are thus stored in Chip 0 (C0). Second, for coarse-grained re-layout, the re-layout unit maps each head to N_{hc} chips. Elements from a certain number of heads are scheduled together to follow the head mapping(blue dashed block). For example, in Fig. 8-b where $N_{hc} = 8$, each burst beat contains elements from a single head after re-layout, so one head is transferred to 8 chips. In Fig. 8-c, elements from 8 heads are placed in one burst beat, so each head is transferred to a single chip. For data onloading, the re-layout performs the reverse process.

VI. Coordinated Cross-device Inference

This section describes CHIME-sys, the CHIME-PIM integrated AFD system with hardware-software co-design to achieve high throughput, addressing the challenges of inter-device data and progress synchronization overheads.

A. Rankset-granular Comm-Comp Overlapping

Data synchronization between the GPU and CHIME-PIM includes transferring the following data: (1) prefilling KVs (proportional to the lengths of inputs); (2) decoding QKV (a token for each request), and (3) decoding attention results (a token for each request). Ideally, with sub-batch scheduling, the data can be transferred asynchronously, e.g., sending the decoding QKV for a sub-batch when executing the other sub-batch. However, GPU-PIM data communication and attention computation on the CHIME-PIM share the memory buses, which indicates that the communication would block the computation. This blocking prevents asynchronous data transfer, incurring non-negligible overhead since the PCIe

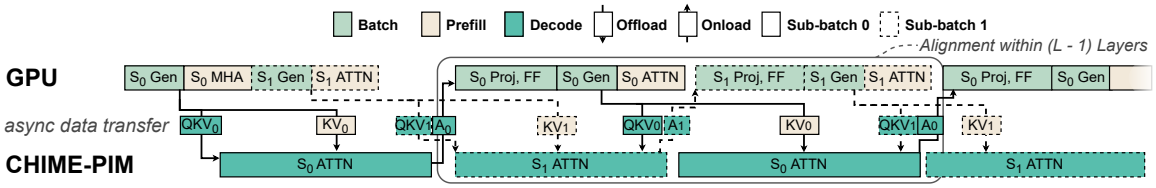


Fig. 10. **Coordinating cross-device inference with CHIME-sys.** *CHIME hides the communication cost and aligns the parallel executions across devices. "ATTN" denotes decoding or prefilling attention, "Gen" denotes QKV Generation and "Proj, FF" denotes the projection and feed-forward operations.*

bandwidth [64] could be orders of magnitude lower than that of CHIME-PIM (Table I).

We identify that the challenge arises from the coarse granularity, i.e., treating all ranks as a whole, for doing either communication or computation. The key to address the challenge is finding the finest granularity of independent and concurrent communication and computation.

Rankset-granular communication-computation overlapping. Our key observation is that, *due to the shared memory buses, only one rank can be accessed in a channel at the same time during communication, while other ranks remain idle.* This motivates our proposal of the *rankset*, which is composed of one rank from every channel, forming the basic granularity of independent communication and computation. In this case, a rankset is the minimum set of ranks to fully utilize all channels. As shown in Fig. 9, each channel has three ranks, forming three ranksets. During the communication of one rankset, other ranksets can perform independent computation without blocking, which could preserve 2/3 of computational power during communication.

Moreover, we achieve *rankset-granular load balance* leveraging the feature of *identical KV cache sizes among layers*. Specifically, we store the KV cache of each request at the granularity of layers in an interleaved manner. It ensures load balance of each rankset transfers, avoiding the rankset transferring the most data slows down the overall progress.

B. Alignment Predicting Scheduling

To prevent idle bubbles caused by synchronizing the progress of parallel operations across the two devices, the key for CHIME's scheduler is to properly select requests to form sub-batches, *aligning the execution latencies of parallel operations on the two devices*. However, LLM's auto-regressive nature makes the alignment challenging, since the execution latencies could vary with factors such as the number of processed tokens, the batch sizes, etc. Prior AFD systems fall short in achieving that goal: for HBM-PIM-based AFD systems [28], [61], the latencies on the PIM side could always be smaller than that on the GPU (as shown in Fig. 2-b). For CPU-based AFD systems [30], they lack modeling the execution latencies of each operation when scheduling. Moreover, the computation time on the CPU side can be interfered by other CPU applications and may lack predictability [17], [53], [54].

Opportunity of performance modeling. To address the challenge, CHIME proposes *Alignment-predicting scheduling*, whose key feature is modeling and predicting the execution latencies on the two devices that helps to align the parallel execution latencies. Leveraging the feature, it selects requests to form sub-batches, whose predicted latencies on the two devices are aligned, as shown in Fig. 10. CHIME exploits the following opportunities to achieve performance predictability. First, the execution with the CHIME-PIM is *interference-free*. Second, the factors that affect the batch performances (e.g., the batch size, number of processed tokens, etc.) are known in advance. Third, abundant prior works have explored methods for predicting the execution on the GPU [14], [23], [77].

C. Implementation Details for Scheduling

Model features. CHIME describes a sub-batch i with following arguments: c_{p_i}, the list of chunk sizes of each prefilling request; f_{p_i}, the list of finished token numbers of each prefilling request; f_{d_i}, the list of finished token numbers (on each rank) of each decoding request. The latencies of both sub-batches can be modeled as follows:

$$T_{\mathrm{GPU}_0} = t_p(c_{p_0}, f_{p_0}) + t_{\mathrm{batch}}(c_{p_0}, f_{d_1}) \quad (2)$$

$$T_{\mathrm{PIM}_0} = t_d(f_{d_0}) + t_{\mathrm{comm}}(f_{d_0}, c_{p_1}) \quad (3)$$

$$T_{\mathrm{GPU}_1} = t_p(c_{p_1}, f_{p_1}) + t_{\mathrm{batch}}(c_{p_1}, f_{d_0}) \quad (4)$$

$$T_{\mathrm{PIM}_1} = t_d(f_{d_1}) + t_{\mathrm{comm}}(f_{d_1}, c_{p_0}) \quad (5)$$

Equation 2,4 denotes the latency on the GPU side, which contains the latency of prefilling attention and batched FC operations. Equation 3,5 denotes the latency on the CHIME-PIM in the granularity of rank, which contains the latency of decoding attention, and overlapped data transfer overheads.

Scheduling policy. With the following steps, the scheduling policy selects requests to form sub-batches with aligned parallel cross-device execution:

1) Add one prefilling request into each sub-batch. When there is no prefilling request, skip the step.
2) For each prefilling request added (which implies an increase in T_{GPU}), CHIME adds N decoding requests to each sub-batch and assign them to ranks in a load balancing manner. After that, it predicts T_{PIM} and T_{GPU} for each sub-batch. If $T_{PIM} < T_{GPU}$, it adds another N decoding requests until $T_{PIM} > T_{GPU}$.
3) If there are remaining prefill requests, it repeats step (1) until the PIM memory is exhausted. When the memory is saturated and the bubble is on the PIM side, the last prefilling request of each sub-batch is chunked,

TABLE I
SIMULATOR DETAILS.

GPU Configuration		ACC Configuration	
Processor	8 A100	**CPU**	2TB + 406GB/s
Capacity	640GB	**HBM-PIM**	640GB + 260.8TB/s (GPU-side) 320GB + 130.4TB/s (Extended)
Bandwidth	16.3TB/s	**DIMM-PIM**	2TB + 1.6TB/s (R-PIM) 2TB + 13.0TB/s (CHIME)
DIMM / DIMM-PIM: DDR4-3200			
Hierarchy	2 Ranks (8 Chips) × 4 Bank Groups × 4 Banks		
DRAM Timing	BL=4:CCD=4:RRD=4/8:RCD=22:RAS=52:RP=22:RC=74: CL=22:WL=16:CDLR=4/12:WR=24:CCDL=8:RTP=12		

* Bank PU's compute-memory ratio N_{cmr}=n for GQA-n.

TABLE II
THE EVALUATED LLM MODELS.

Model	Layers N_l	Heads N_h	Embedding D_e	Type	TP	DP
OPT-66B	64	72	9216	MHA	2	4
QWEN-72B	80	64	8192	GQA-8	4	2
GPT-175B	96	96	12288	MHA	8	1

* Precision: FP16; Head Embedding: $E_h = D_e/N_h = 128$.

TABLE III
REAL-WORLD TRACES.

Trace	Avg. L_{in}	Std. L_{in}	Avg. L_{out}	Std. L_{out}
OpenR1-Math-220k [5]	96.0	75.1	12684.1	8464.6
Dolphin-r1 [2]	201.9	563.0	3926.2	4216.0
OpenThoughts-114k-math [6]	89.4	66.7	6366.7	4662.9

dynamically adjusting T_{GPU} to make it as close as possible to the T_{PIM} of the other sub-batch.

The larger the value of N, the coarser the granularity of batch execution time adjustment, but the more beneficial it is for achieving load balance among ranks for requests. In our implementation, since MHA has more heads and these heads can be evenly distributed across chips, the N for MHA is set to 1, while the N for GQA is set to 16.

Model selection. Now we describe how CHIME models the execution latencies of various inference operations. First, to model T_{GPU}, we leverage Random Forest Regression (RFR) for its three advantages: capability of incremental learning, low latency, and high accuracy. The efficiency of RFR for execution latency prediction has been proven in many prior works [53], [80], and it is open to use other models that feature similarly. Second, to model T_{PIM}, given that CHIME-PIM execution is featured predictable (i.e., execution time is linearly related to the number of computed/transferred tokens), we can use a simple and fast linear model for t_{comm} and each rank's t_d. The overall t_d is determined by the rank with the longest execution time. The models can be described with the following formula (take sub-batch 0 as an example):

$$T_{\mathrm{PIM}_0} = \mathrm{Linear}(\sum f_{d_0}, \mathrm{len}(f_{d_0}), \sum c_{p_1})$$

$$T_{\mathrm{GPU}_0} = \mathrm{RFR}_p(c_{p_0}, f_{p_0}) + \mathrm{RFR}_{\mathrm{batch}}(\sum c_{p_0}, \mathrm{len}(f_{d_0}))$$

Runtime profiling. CHIME applies runtime profiling, which collects the latency information and corresponding batch information during inference. The dataset maintained by CHIME dynamically evolves with the collected data for incrementally updating the model, thereby adapting to changing execution environment. Moreover, we evaluate the prediction models of the CHIME scheduler by splitting the collected 1000 data points into training and test sets in an 8:2 ratio. The experimental results show that our model exhibits great performance predictability, achieving prediction relative errors of less than ∼1% (median value $< 0.5\%$).

VII. EVALUATION

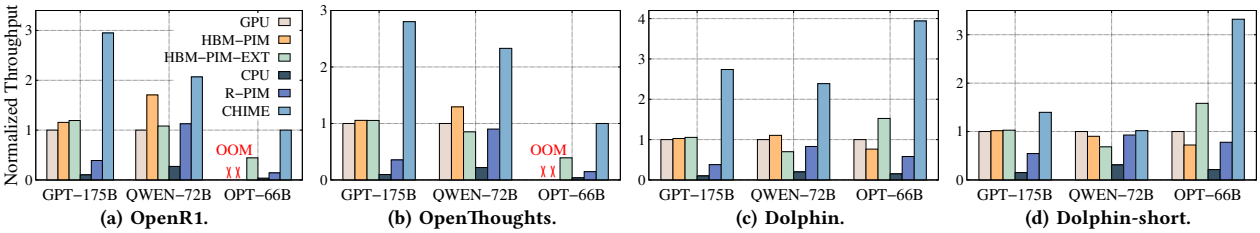
A. Methodology

Experimental setup. Following prior works [12], [28], [69], our goal is achieving *high inference throughput* in the long-context and decoding-dominant scenario. The evaluation is built upon DGX-A100 [57]. On the GPU side, 8 NVIDIA A100 GPUs, each with 5 HBM2e of 80 GB capacity, are integrated, providing a total of 156 TFLOPs on FP16. The GPUs are connected via NVLink [74]. On the CPU side, there are 16 channels with 2 DIMMs of total 2TB capacity, equipped with the proposed PIM. We develop a simulator integrating (1) AttAcc [61], the roofline-based simulation for GPU; and (2) CHIME-PIM-sim based on modified DRAMSim3 [51], the trace-driven simulation for CHIME-PIM. We add PIM commands, related timing constraints, and FIFO-based scheduling methods for cycle-level simulation.

Baseline systems. We compare CHIME with one GPU baseline and four AFD baseline systems: (1) *GPU-only*, which executes LLM inference exclusively on GPUs. (2) *GPU with HBM-PIM*, which equips all existing GPU HBMs with bank-level PUs. (3) *GPU with HBM-PIM-EXT*, which equips with extended disaggregated HBM-PIMs. Compared to the "HBM-PIM" baseline, the HBM-PIMs in "HBM-PIM-EXT" do not store model parameters. (4) *GPU with rank-level DIMM-PIM* (R-PIM), which equips all DIMMs with rank-level PUs. (5) *GPU with CPU offloading*, which leverages CPU as the accelerator. Our simulations of baseline systems uniformly assume maximal accelerator-side bandwidth utilization during attention computation. Table I lists the hardware specifications. For GPU, CPU, and DIMM(-PIM) configurations, we follow the configurations in DGX-A100 systems. For HBM-PIM-EXT, considering that HBM2e is more than $6\times$ the price of DDR4 [3], [4], [7], we configure 20 HBM-PIM modules (320 GB) to provide $\sim 1/6$ of the capacity available in CHIME.

LLM models. We evaluate CHIME on three LLM models with varying model sizes: OPT-66B, QWEN-72B, and GPT-175B, with FP16 as the data precision. Considering both the GPU memory capacity and inter-GPU communication overhead, we apply various parallelism configurations, listed in Table II.

Workloads. We use three real-world LLM inference datasets: OpenR1-Math-220K [5], OpenThoughts-114k-math [6], Dolphin-r1 [2]. All of these datasets are used



(a) OpenR1. **(b) OpenThoughts.** **(c) Dolphin.** **(d) Dolphin-short.**

Fig. 11. **End-to-end inference throughput on real-world traces.** *The throughput of GPU-only baseline is normalized to 1. For the OPT-66B model, GPU out-of-memory (OOM) errors occurred in two traces, so the throughput of CHIME is normalized to 1. In case (d), length of requests is reduced by 10×.*

for evaluating the performance of LLM inference on long context. The details about the traces are shown in Table III. Further, to evaluate the short context scenario, we add an additional trace, Dolphin-short, which shortens each text entry in Dolphin-r1 to 1/10 of its original length.

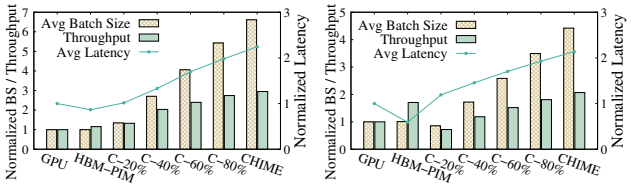
B. End-to-end Performance

We evaluate the end-to-end throughput for each LLM model and workload using various baselines. For each evaluation, we randomly select and execute 1,000 requests from the traces. We measure the overall throughput by dividing the total number of output tokens by the total execution time.

Throughput with real-world traces. Fig. 11-a,b,c presents the normalized throughput of CHIME compared with five baselines. The results show that CHIME achieves the highest throughput over all baselines with various settings, up to 5.15× higher throughput than the HBM-PIM baseline, 3.45× than the HBM-PIM-EXT baseline, 3.94× than the GPU-only baseline, and 7.21× higher than R-PIM baseline. These improvements are attributed to the following reasons. First, CHIME achieves much larger batch sizes compared to the HBM-based baselines. For example, for GPT-175B, CHIME has 2 TB of memory on the host memory for KV cache storage, while GPU and HBM-PIM baselines have only about 310 GB of memory in total after deploying the model, and HBM-PIM-EXT has only 320GB of memory under the same cost budget. Second, the CPU and R-PIM baselines suffer from the limited bandwidth. For example, CHIME offers about 8× the bandwidth of R-PIM with bank-level PUs.

We observe that HBM-PIM's sub-batch method demonstrates promising efficiency in achieving high throughput with enhanced parallelism, though our analysis reveals certain limitations when processing Dolphin trace on OPT-66B. Dividing a batch into sub-batches results in smaller batch sizes, leading to severe GPU underutilization and performance below that of the conventional GPU-only implementation. On the contrary, CHIME benefits from much larger batch sizes and thus achieves higher throughput.

Performance improvement breakdown. We analyze the sources of performance improvement compared with the GPU and HBM-PIM baselines. Fig.12 shows the throughput, average batch size (the sum of two sub-batches), and average latency per batch from our end-to-end evaluation. Compared with GPU and HBM-PIM, CHIME increases the batch size



(a) GPT-175B. **(b) QWEN-72B.**

Fig. 12. **Breakdown of performance improvement with CHIME.** *The results are evaluated using the OpenR1 trace. "C-N%" denotes CHIME with N% available memory capacity. "BS" denotes "batch size". "Latency" is "latency per batch". The results of GPU baseline are normalized to 1*

by 6.6×, while the latency per batch increases by only 2.2×. This indicates that the throughput improvement of CHIME primarily stems from the increased batch size and the corresponding improvement in GPU utilization.

In addition, Fig. 12 illustrates the trend of how batch size growth affects throughput. We manually limit the memory capacity available to CHIME and measure the resulting performance. As the available memory gradually increases from 10% to 100%, the batch size scales linearly. However, since the latency per batch also increases, the rate of throughput improvement gradually diminishes. This observation is consistent with the analysis in Fig. 2, which shows that the marginal benefit of increasing batch size on throughput decreases.

Performance in short context scenarios. As shown in Fig. 11-d, in the short context scenario, CHIME still achieves higher throughput than both the GPU and HBM-PIM baselines by achieving higher batch sizes. However, compared to Fig. 11-c, the advantage of CHIME on each model is diminished. This is because in the short context scenario, the marginal benefit of increasing the batch size exhibits diminishing returns. This observation is also consistent with our analysis in §III-D.

Scalability analysis. We further evaluate CHIME performance with various CHIME-PIM configurations, showing that performance scalability requires simultaneous scaling in memory capacity and bandwidth. We first establish the base memory configuration as "1 rankset, 512 GB". Based on this, we scale up the memory in different ways for different baselines: "Bw-only" indicates that we only scale up the memory bandwidth with more ranksets but keep the capacity at 512GB, while "Cap-only" means we only scale up

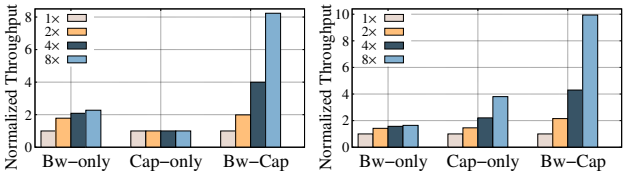


Fig. 13. **Scalability analysis.** *The results are evaluated using the OpenR1 trace. The performance of the base memory configuration is normalized to 1.*

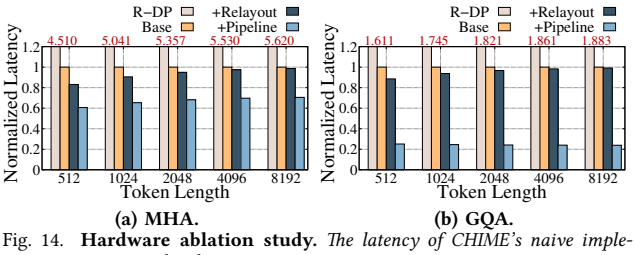


Fig. 14. **Hardware ablation study.** *The latency of CHIME's naive implementation is normalized to 1.*

the capacity from 512GB to 4TB. "Bw-Cap" implies that we scale up both the bandwidth and capacity. We execute 1,000 requests from the trace and calculate the average throughput.

Fig. 13 shows the results of GPT-175B and QWEN-72B with OpenR1 trace. Results for other models and traces are similar. It shows that expanding either bandwidth or capacity alone does not effectively improve throughput. For example, when exclusively scaling memory capacity or bandwidth by 8× on GPT-175B, the throughput only increases by 2.28× and 1.01×, respectively. In contrast, when both bandwidth and capacity are enlarged, the throughput increases by 8.23×. This demonstrates that CHIME effectively leverages the value of both scalability aspects.

C. Ablation Study

Ablation study for CHIME-PIM. We analyze our hardware optimizations for: (1) bubble-free pipelining (§V-A); (2) hybrid re-layout (§V-B). We implement a baseline bank-level CHIME-PIM that maps head computation across chips. Following prior work [68], we conservatively estimate the CPU-side re-layout cost as the memory access time required to read data stored with one layout and write it into another layout. Performance results with various token lengths are shown in Fig. 14. First, the bubble-free pipelining achieves about 27.9% and 74.4% latency reduction on MHA and GQA computation, respectively, which is attributed to the overlapping and specific head mapping methods. Second, the hardware hybrid re-layout enables up to 17% latency reduction. The reason why the proportion of gain decreases as token length increases is that the attention computation gradually dominates the overall execution time. Nonetheless, we identify that it is still needed since the re-layout overheads accumulate in each layer of each token generation. With all hardware optimizations, CHIME shows 1.42–4.18× speedup. In addition, all bank-level implementations achieve over 1.5× speedup than the rank-level DIMM-PIM (R-DP).

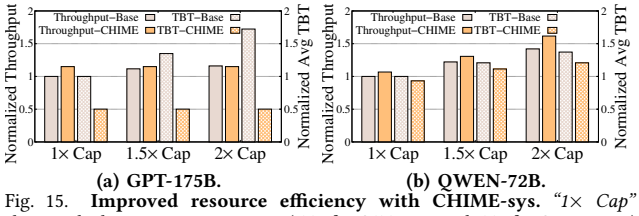


Fig. 15. **Improved resource efficiency with CHIME-sys.** *"1× Cap" denotes the basic memory capacity (2TB for GPT-175B and 1TB for QWEN-72B) according to Table II. The results are evaluated using the OpenR1 trace.*

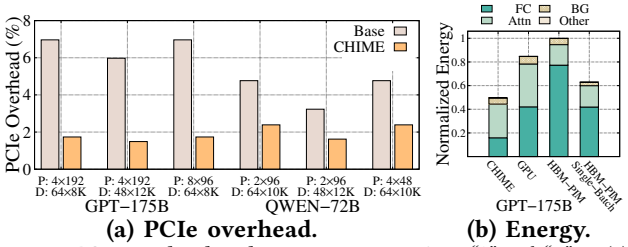


Fig. 16. **PCIe overhead and energy consumption.** *"P" and "D" in (a) denotes the batch sizes and token lengths of prefilling and decoding requests in a batch. The energy consumption of "GPU" in (b) is normalized to 1.*

Ablation study for scheduler. We evaluate how CHIME's scheduler effectively improves the resource utilization. Fig. 15 shows the average throughput and Time-between-Tokens (TBTs) of different scheduling methods on the left and right y-axes, respectively. The baseline represents the scheduling policy that prioritizes filling the capacity of CHIME-PIM, while CHIME denotes the alignment-predicting scheduling. The results show that CHIME's scheduler can significantly reduce latency by up to 70.93% without sacrificing the throughput, showing its ability of eliminating idle bubbles on the GPU side. CHIME even slightly improves the throughput by aligning the batches with prefilling requests and achieving better load balancing. Moreover, for the MHA model, if we increase the capacity of CHIME-PIM, the baseline selects larger batch sizes and causes higher TBTs without improving the throughput (since attention becomes the bottleneck), while CHIME can avoid generating bubbles and prevent the growth of TBT. CHIME performs better with MHA models, because with the GQA model, achieving the attention bottleneck requires selecting requests that occupy larger memory capacity, or it may not even achieve the attention bottleneck.

PCIe overhead analysis. We evaluate how the cross-device PCIe communication degrades the throughput with or without rankset-granular communication computation overlapping. As shown in Fig. 16, applying the optimization could reduce the overhead by up to 75.08% with different batch configurations. This indicates that with the 4 ranksets of DGX-A100, most PCIe overheads are hidden with independent communication and computation at the rankset granularity.

D. Cost Analysis

Energy consumption. We evaluate the energy consumption of CHIME with GPU and HBM-PIM. For GPU-side energy, we

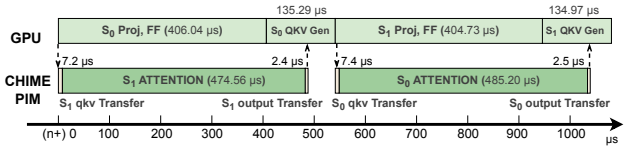


Fig. 17. **Decode execution timeline of CHIME pipeline for single LLM layer.** *The batch is executed with GPT-175B and the OpenR1 trace.*

follow the evaluation methods in AttAcc simulators. For PIM-side energy, first, we use power parameters (e.g., VDD, IDD) and timing parameters (e.g., tRAS, tRP) to calculate energy of ACT, RD, WR, and so on [55], [56]. Second, we follow the scaling methods [59], [61], [62] to calculate read/write energy (e.g., MAC, Load shared buffer) inside the DRAM chips (or DRAM die); Third, we obtain the chip I/O energy from CACTI-IO [31]. We provide three baselines: GPU, HBM-PIM, and HBM-PIM without sub-batch scheduling.

Fig. 16-b details the energy breakdown for GPT-175B on openR1 traces, where FC and attention computations dominate. First, CHIME achieves a 40% total energy reduction, primarily by executing FC on the GPU with larger batch sizes, which reduces the number of weight loading. Second, HBM-PIM with sub-batch scheduling consumes more energy, since it halves the batch size. Third, for attention computation, CHIME-PIM's multi-chip distributed computation consumes energy comparable to the GPU but higher than HBM-PIM.

Hardware overhead. We evaluate the area overhead of bank PUs, whose in-DRAM-chip integration directly impacts memory capacity. Assuming command decoding and data paths reuse existing DRAM logic, we synthesize the design using a TSMC 28nm process. A single bank PU and the shared buffer consume 0.0032 mm^2 and 0.051 mm^2, respectively. Accounting for density differences between logic and DRAM processes, we estimate that implementation in a 1z-nm DRAM node would roughly double the synthesized area. While efficiently supporting GQA requires scaling the bank PU proportionally, we consider this overhead highly acceptable for two reasons. First, the massive capacity of DIMMs, effectively amortizes the area cost. Further, emerging 3D integration technologies can further mitigate or eliminate this die area penalty.

Timeline breakdown analysis. As shown in Fig. 17, we extract a representative batch from the actual execution of the end-to-end evaluation as an example and perform a detailed timeline breakdown analysis. We observe that with our designs, the computation on the GPU and PIM sides is effectively pipelined with minor bubbles.

VIII. Related Work

PIM/PNM architecture for LLM inference. In addition to PIM designs [28], [61] leveraged in AFD systems for batched inference, IANUS [50], [67] accelerates non-batch inference with distinct mapping methods. For long context scenarios, LoL-PIM [40], CXL-PNM [63] and HPU [66] propose scalable capacity and bandwidth, in which DRM can guide their configurations to achieve optimal performance. Duplex [79] targets Mixture of Experts (MoE) inference. The DRM can be readily extended to analyze MoE models, as their FC layers exhibit performance patterns similar to those of dense models. Papi [27] proposes dynamic scheduling methods on various FC devices under different batch sizes. We identify that it is orthogonal to our work, and these devices represent different Line-FCs in the DRM model, which show high performance potential under several real-world scenarios, such as low request per second. CENT [22] proposes scalable PIM-only inference and could achieve high performance in various scenarios. In AFD inference scenarios, the cost efficiency of its GDDR6-PIM degrades, as shown in Fig. 3.

Systems with enhanced capacity for KV cache. FastDecode [25] and NEO [30] offload the KV cache to the host memory and compute attention with the CPU. InstAttention [60] offloads attention with computational storage drives, utilizing the internal bandwidth of the SSD. These systems suffer limited bandwidth for attention computation, as analyzed in §III-A. Other works such as CachedAttention [20] and FlexGen [69] utilizes host memory to store the KV cache, and fetch it back to GPU for attention computation. Their latencies is limited by the PCIe bandwidth.

GPU-only AFD systems. Some prior works disaggregate Attention and FC on heterogeneous GPUs [71], [72], [83]. Our DRM-based methodology can help them more effectively balance compute between Attention and FC. While GPUs offer superior generality and software compatibility, DIMM-PIM provides substantial cost advantages.

IX. Conclusion

This paper presents the first general AFD performance model, Disaggregated Roofline Model, from which we conclude the "Liebig's Law" that guides the design of CHIME, a hardware-software co-designed AFD system with DIMM-PIM that offers scalable memory capacity and bandwidth for LLM inference. CHIME can enable efficient DIMM-PIM attention computation and maximizes the resource utilization for cross-device inference. Evaluations show CHIME achieves significantly higher throughput, establishing a new paradigm for efficient long-context LLM inference.

Acknowledgments

We thank the anonymous ISCA reviewers for their constructive feedback. This work was supported in part by the National Natural Science Foundation of China (No. 62302300, 62432010, 62472279, and U24B20165), the Fundamental and Interdisciplinary Disciplines Breakthrough Plan of the Ministry of Education of China (JYB2025XDXM122), the Fundamental Research Funds for the Central Universities, and the Advanced Research Center for Agent-Oriented OS (TC20260106010). Corresponding authors: Dong Du (dd_nirvana@sjtu.edu.cn) and Naifeng Jing (sjtuj@sjtu.edu.cn).

References

[1] "Nvidia readying h20 ai gpu for chinese market," https://www.techpowerup.com/318595/nvidia-readying-h20-ai-gpu-for-chinese-market, 2024.

[2] "cognitivecomputations/dolphin-r1 · datasets at hugging face," https://huggingface.co/datasets/cognitivecomputations/dolphin-r1, 2025, referenced April 2025.

[3] "Keepa: search for ddr4 lrdimm," https://keepa.com/#!search/1-DDR42025.

[4] "Keepa: search for ddr5 rdimm," https://keepa.com/?&#!search/1-DDR5

[5] "open-r1/openr1-math-220k · datasets at hugging face," https://huggingface.co/datasets/open-r1/OpenR1-Math-220k, 2025, referenced April 2025.

[6] "open-r1/openthoughts-114k-math · datasets at hugging face," https://huggingface.co/datasets/open-r1/OpenThoughts-114k-math, 2025, referenced April 2025.

[7] "Inside nvidia grace cpu: Nvidia amps up superchip engineering for hpc and ai," https://developer.nvidia.com/blog/inside-nvidia-grace-cpu-nvidia-amps-up-superchip-engineering-for-hpc-and-ai/, 2026.

[8] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulavani, A. Tumanov, and R. Ramjee, "Taming throughput-latency tradeoff in llm inference with sarathi-serve," in *Proceedings of the 18th USENIX Conference on Operating Systems Design and Implementation*, ser. OSDI'24. USA: USENIX Association, 2024.

[9] R. Anil, A. M. Dai, O. Firat, M. Johnson, D. Lepikhin, A. Passos, S. Shakeri, E. Taropa, P. Bailey, Z. Chen, E. Chu, J. H. Clark, L. E. Shafey, Y. Huang, K. Meier-Hellstern, G. Mishra, E. Moreira, M. Omernick, K. Robinson, S. Ruder, Y. Tay, K. Xiao, Y. Xu, Y. Zhang, G. H. Abrego, J. Ahn, J. Austin, P. Barham, J. Botha, J. Bradbury, S. Brahma, K. Brooks, M. Catasta, Y. Cheng, C. Cherry, C. A. Choquette-Choo, A. Chowdhery, C. Crepy, S. Dave, M. Dehghani, S. Dev, J. Devlin, M. Díaz, N. Du, E. Dyer, V. Feinberg, F. Feng, V. Fienber, M. Freitag, X. Garcia, S. Gehrmann, L. Gonzalez, G. Gur-Ari, S. Hand, H. Hashemi, L. Hou, J. Howland, A. Hu, J. Hui, J. Hurwitz, M. Isard, A. Ittycheriah, M. Jagielski, W. Jia, K. Kenealy, M. Krikun, S. Kudugunta, C. Lan, K. Lee, B. Lee, E. Li, M. Li, W. Li, Y. Li, J. Li, H. Lim, H. Lin, Z. Liu, F. Liu, M. Maggioni, A. Mahendru, J. Maynez, V. Misra, M. Moussalem, Z. Nado, J. Nham, E. Ni, A. Nystrom, A. Parrish, M. Pellat, M. Polacek, A. Polozov, R. Pope, S. Qiao, E. Reif, B. Richter, P. Riley, A. C. Ros, A. Roy, B. Saeta, R. Samuel, R. Shelby, A. Slone, D. Smilkov, D. R. So, D. Sohn, S. Tokumine, D. Valter, V. Vasudevan, K. Vodrahalli, X. Wang, P. Wang, Z. Wang, T. Wang, J. Wieting, Y. Wu, K. Xu, Y. Xu, L. Xue, P. Yin, J. Yu, Q. Zhang, S. Zheng, C. Zheng, W. Zhou, D. Zhou, S. Petrov, and Y. Wu, "Palm 2 technical report," 2023. [Online]. Available: https://arxiv.org/abs/2305.10403

[10] H. Asghari-Moghaddam, Y. H. Son, J. H. Ahn, and N. S. Kim, "Chameleon: versatile and practical near-dram acceleration architecture for large memory systems," in *The 49th Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO-49. IEEE Press, 2016.

[11] Y. Bai, X. Lv, J. Zhang, H. Lyu, J. Tang, Z. Huang, Z. Du, X. Liu, A. Zeng, L. Hou, Y. Dong, J. Tang, and J. Li, "Longbench: A bilingual, multitask benchmark for long context understanding," 2024. [Online]. Available: https://arxiv.org/abs/2308.14508

[12] S. Cao, S. Liu, T. Griggs, P. Schafhalter, X. Liu, Y. Sheng, J. E. Gonzalez, M. Zaharia, and I. Stoica, "Moe-lightning: High-throughput moe inference on memory-constrained gpus," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, ser. ASPLOS '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 715–730. [Online]. Available: https://doi.org/10.1145/3669940.3707267

[13] L. Chen, D. Lyu, J. Jiang, Q. Wang, Z. Mao, and N. Jing, "Asyncdimm: Achieving asynchronous execution in dimm-based near-memory processing," in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2025, pp. 518–532.

[14] W. Cui, H. Zhao, Q. Chen, N. Zheng, J. Leng, J. Zhao, Z. Song, T. Ma, Y. Yang, C. Li, and M. Guo, "Enable simultaneous dnn services based on deterministic operator overlap and precise latency prediction," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, ser. SC '21. New York, NY, USA: Association for Computing Machinery, 2021. [Online]. Available: https://doi.org/10.1145/3458817.3476143

[15] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "Flashattention: fast and memory-efficient exact attention with io-awareness," in *Proceedings of the 36th International Conference on Neural Information Processing Systems*, ser. NIPS '22. Red Hook, NY, USA: Curran Associates Inc., 2022.

[16] DeepSeek-AI, D. Guo, D. Yang, H. Zhang, J. Song, R. Zhang, R. Xu, Q. Zhu, S. Ma, P. Wang, X. Bi, X. Zhang, X. Yu, Y. Wu, Z. F. Wu, Z. Gou, Z. Shao, Z. Li, Z. Gao, A. Liu, B. Xue, B. Wang, B. Wu, B. Feng, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan, D. Dai, D. Chen, D. Ji, E. Li, F. Lin, F. Dai, F. Luo, G. Hao, G. Chen, G. Li, H. Zhang, H. Bao, H. Xu, H. Wang, H. Ding, H. Xin, H. Gao, H. Qu, H. Li, J. Guo, J. Li, J. Wang, J. Chen, J. Yuan, J. Qiu, J. Li, J. L. Cai, J. Ni, J. Liang, J. Chen, K. Dong, K. Hu, K. Gao, K. Guan, K. Huang, K. Yu, L. Wang, L. Zhang, L. Zhao, L. Wang, L. Zhang, L. Xu, L. Xia, M. Zhang, M. Zhang, M. Tang, M. Li, M. Wang, M. Li, N. Tian, P. Huang, P. Zhang, Q. Wang, Q. Chen, Q. Du, R. Ge, R. Zhang, R. Pan, R. Wang, R. J. Chen, R. L. Jin, R. Chen, S. Lu, S. Zhou, S. Chen, S. Ye, S. Wang, S. Yu, S. Zhou, S. Pan, S. S. Li, S. Zhou, S. Wu, S. Ye, T. Yun, T. Pei, T. Sun, T. Wang, W. Zeng, W. Zhao, W. Liu, W. Liang, W. Gao, W. Yu, W. Zhang, W. L. Xiao, W. An, X. Liu, X. Wang, X. Chen, X. Nie, X. Cheng, X. Liu, X. Xie, X. Liu, X. Yang, X. Li, X. Su, X. Lin, X. Q. Li, X. Jin, X. Shen, X. Chen, X. Sun, X. Wang, X. Song, X. Zhou, X. Wang, X. Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Y. Zhang, Y. Xu, Y. Li, Y. Zhao, Y. Sun, Y. Wang, Y. Yu, Y. Zhang, Y. Shi, Y. Xiong, Y. He, Y. Piao, Y. Wang, Y. Tan, Y. Ma, Y. Liu, Y. Guo, Y. Ou, Y. Wang, Y. Gong, Y. Zou, Y. He, Y. Xiong, Y. Luo, Y. You, Y. Liu, Y. Zhou, Y. X. Zhu, Y. Xu, Y. Huang, Y. Li, Y. Zheng, Y. Zhu, Y. Ma, Y. Tang, Y. Zha, Y. Yan, Z. Z. Ren, Z. Ren, Z. Sha, Z. Fu, Z. Xu, Z. Xie, Z. Zhang, Z. Hao, Z. Ma, Z. Yan, Z. Wu, Z. Gu, Z. Zhu, Z. Liu, Z. Li, Z. Xie, Z. Song, Z. Pan, Z. Huang, Z. Xu, Z. Zhang, and Z. Zhang, "Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning," 2025. [Online]. Available: https://arxiv.org/abs/2501.12948

[17] C. Delimitrou and C. Kozyrakis, "Paragon: Qos-aware scheduling for heterogeneous datacenters," *SIGPLAN Not.*, vol. 48, no. 4, p. 77–88, Mar. 2013. [Online]. Available: https://doi.org/10.1145/2499368.2451125

[18] F. Devaux, "The true processing in memory accelerator," in *2019 IEEE Hot Chips 31 Symposium (HCS)*, 2019, pp. 1–24.

[19] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," 2019. [Online]. Available: https://arxiv.org/abs/1810.04805

[20] B. Gao, Z. He, P. Sharma, Q. Kang, D. Jevdjic, J. Deng, X. Yang, Z. Yu, and P. Zuo, "Cost-Efficient large language model serving for multi-turn conversations with CachedAttention," in *2024 USENIX Annual Technical Conference (USENIX ATC 24)*. Santa Clara, CA: USENIX Association, Jul. 2024, pp. 111–126. [Online]. Available: https://www.usenix.org/conference/atc24/presentation/gao-bin-cost

[21] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Vaughan, A. Yang, A. Fan, A. Goyal, A. Hartshorn, A. Yang, A. Mitra, A. Sravankumar, A. Korenev, A. Hinsvark, A. Rao, A. Zhang, A. Rodriguez, A. Gregerson, A. Spataru, B. Roziere, B. Biron, B. Tang, B. Chern, C. Caucheteux, C. Nayak, C. Bi, C. Marra, C. McConnell, C. Keller, C. Touret, C. Wu, C. Wong, C. C. Ferrer, C. Nikolaidis, D. Allonsius, D. Song, D. Pintz, D. Livshits, D. Wyatt, D. Esiobu, D. Choudhary, D. Mahajan, D. Garcia-Olano, D. Perino, D. Hupkes, E. Lakomkin, E. AlBadawy, E. Lobanova, E. Dinan, E. M. Smith, F. Radenovic, F. Guzmán, F. Zhang, G. Synnaeve, G. Lee, G. L. Anderson, G. Thattai, G. Nail, G. Mialon, G. Pang, G. Cucurell, H. Nguyen, H. Korevaar, H. Xu, H. Touvron, I. Zarov, I. A. Ibarra, I. Kloumann, I. Misra, I. Evtimov, J. Zhang, J. Copet, J. Lee, J. Geffert, J. Vranes, J. Park, J. Mahadeokar, J. Shah, J. van der Linde, J. Billock, J. Hong, J. Lee, J. Fu, J. Chi, J. Huang, J. Liu, J. Wang, J. Yu, J. Bitton, J. Spisak, J. Park, J. Rocca, J. Johnstun, J. Saxe, J. Jia, K. V. Alwala, K. Prasad, K. Upasani, K. Plawiak, K. Li, K. Heafield, K. Stone, K. El-Arini, K. Iyer, K. Malik, K. Chiu, K. Bhalla, K. Lakhotia, L. Rantala-Yeary, L. van der Maaten, L. Chen, L. Tan, L. Jenkins, L. Martin, L. Madaan, L. Malo, L. Blecher, L. Landzaat, L. de Oliveira, M. Muzzi, M. Pasupuleti, M. Singh, M. Paluri, M. Kardas, M. Tsimpoukelli, M. Oldham, M. Rita, M. Pavlova, M. Kambadur, M. Lewis, M. Si, M. K. Singh, M. Hassan, N. Goyal, N. Torabi, N. Bashlykov, N. Bogoychev, N. Chatterji, N. Zhang, O. Duchenne, O. Çelebi, P. Alrassy, P. Zhang, P. Li, P. Vasic, P. Weng, P. Bhargava, P. Dubal, P. Krishnan, P. S. Koura, P. Xu, Q. He, Q. Dong, R. Srinivasan, R. Ganapathy, R. Calderer, R. S.

Cabral, R. Stojnic, R. Raileanu, R. Maheswari, R. Girdhar, R. Patel, R. Sauvestre, R. Polidoro, R. Sumbaly, R. Taylor, R. Silva, R. Hou, R. Wang, S. Hosseini, S. Chennabasappa, S. Singh, S. Bell, S. S. Kim, S. Edunov, S. Nie, S. Narang, S. Raparthy, S. Shen, S. Wan, S. Bhosale, S. Zhang, S. Vandenhende, S. Batra, S. Whitman, S. Sootla, S. Collot, S. Gururangan, S. Borodinsky, T. Herman, T. Fowler, T. Sheasha, T. Georgiou, T. Scialom, T. Speckbacher, T. Mihaylov, T. Xiao, U. Karn, V. Goswami, V. Gupta, V. Ramanathan, V. Kerkez, V. Gonguet, V. Do, V. Vogeti, V. Albiero, V. Petrovic, W. Chu, W. Xiong, W. Fu, W. Meers, X. Martinet, X. Wang, X. Wang, X. E. Tan, X. Xia, X. Xie, X. Jia, X. Wang, Y. Goldschlag, Y. Gaur, Y. Babaei, Y. Wen, Y. Song, Y. Zhang, Y. Li, Y. Mao, Z. D. Coudert, Z. Yan, Z. Chen, Z. Papakipos, A. Singh, A. Srivastava, A. Jain, A. Kelsey, A. Shajnfeld, A. Gangidi, A. Victoria, A. Goldstand, A. Menon, A. Sharma, A. Boesenberg, A. Baevski, A. Feinstein, A. Kallet, A. Sangani, A. Teo, A. Yunus, A. Lupu, A. Alvarado, A. Caples, A. Gu, A. Ho, A. Poulton, A. Ryan, A. Ramchandani, A. Dong, A. Franco, A. Goyal, A. Saraf, A. Chowdhury, A. Gabriel, A. Bharambe, A. Eisenman, A. Yazdan, B. James, B. Maurer, B. Leonhardi, B. Huang, B. Loyd, B. D. Paola, B. Paranjape, B. Liu, B. Wu, B. Ni, B. Hancock, B. Wasti, B. Spence, B. Stojkovic, B. Gamido, B. Montalvo, C. Parker, C. Burton, C. Mejia, C. Liu, C. Wang, C. Kim, C. Zhou, C. Hu, C.-H. Chu, C. Cai, C. Tindal, C. Feichtenhofer, C. Gao, D. Civin, D. Beaty, D. Kreymer, D. Li, D. Adkins, D. Xu, D. Testuggine, D. David, D. Parikh, D. Liskovich, D. Foss, D. Wang, D. Le, D. Holland, E. Dowling, E. Jamil, E. Montgomery, E. Presani, E. Hahn, E. Wood, E.-T. Le, E. Brinkman, E. Arcaute, E. Dunbar, E. Smothers, F. Sun, F. Kreuk, F. Tian, F. Kokkinos, F. Ozgenel, F. Caggioni, F. Kanayet, F. Seide, G. M. Florez, G. Schwarz, G. Badeer, G. Swee, G. Halpern, G. Herman, G. Sizov, Guangyi, Zhang, G. Lakshminarayanan, H. Inan, H. Shojanazeri, H. Zou, H. Wang, H. Zha, H. Habeeb, H. Rudolph, H. Suk, H. Aspegren, H. Goldman, H. Zhan, I. Damlaj, I. Molybog, I. Tufanov, I. Leontiadis, I.-E. Veliche, I. Gat, J. Weissman, J. Geboski, J. Kohli, J. Lam, J. Asher, J.-B. Gaya, J. Marcus, J. Tang, J. Chan, J. Zhen, J. Reizenstein, J. Teboul, J. Zhong, J. Jin, J. Yang, J. Cummings, J. Carvill, J. Shepard, J. McPhie, J. Torres, J. Ginsburg, J. Wang, K. Wu, K. H. U, K. Saxena, K. Khandelwal, K. Zand, K. Matosich, K. Veeraraghavan, K. Michelena, K. Li, K. Jagadeesh, K. Huang, K. Chawla, K. Huang, L. Chen, L. Garg, L. A, L. Silva, L. Bell, L. Zhang, L. Guo, L. Yu, L. Moshkovich, L. Wehrstedt, M. Khabsa, M. Avalani, M. Bhatt, M. Mankus, M. Hasson, M. Lennie, M. Reso, M. Groshev, M. Naumov, M. Lathi, M. Keneally, M. Liu, M. L. Seltzer, M. Valko, M. Restrepo, M. Patel, M. Vyatskov, M. Samvelyan, M. Clark, M. Macey, M. Wang, M. J. Hermoso, M. Metanat, M. Rastegari, M. Bansal, N. Santhanam, N. Parks, N. White, N. Bawa, N. Singhal, N. Egebo, N. Usunier, N. Mehta, N. P. Laptev, N. Dong, N. Cheng, O. Chernoguz, O. Hart, O. Salpekar, O. Kalinli, P. Kent, P. Parekh, P. Saab, P. Balaji, P. Rittner, P. Bontrager, P. Roux, P. Dollar, P. Zvyagina, P. Ratanchandani, P. Yuvraj, Q. Liang, R. Alao, R. Rodriguez, R. Ayub, R. Murthy, R. Nayani, R. Mitra, R. Parthasarathy, R. Li, R. Hogan, R. Battey, R. Wang, R. Howes, R. Rinott, S. Mehta, S. Siby, S. J. Bondu, S. Datta, S. Chugh, S. Hunt, S. Dhillon, S. Sidorov, S. Pan, S. Mahajan, S. Verma, S. Yamamoto, S. Ramaswamy, S. Lindsay, S. Lindsay, S. Feng, S. Lin, S. C. Zha, S. Patil, S. Shankar, S. Zhang, S. Zhang, S. Wang, S. Agarwal, S. Sajuyigbe, S. Chintala, S. Max, S. Chen, S. Kehoe, S. Satterfield, S. Govindaprasad, S. Gupta, S. Deng, S. Cho, S. Virk, S. Subramanian, S. Choudhury, S. Goldman, T. Remez, T. Glaser, T. Best, T. Koehler, T. Robinson, T. Li, T. Zhang, T. Matthews, T. Chou, T. Shaked, V. Vontimitta, V. Ajayi, V. Montanez, V. Mohan, V. S. Kumar, V. Mangla, V. Ionescu, V. Poenaru, V. T. Mihailescu, V. Ivanov, W. Li, W. Wang, W. Jiang, W. Bouaziz, W. Constable, X. Tang, X. Wu, X. Wang, X. Wu, X. Gao, Y. Kleinman, Y. Chen, Y. Hu, Y. Jia, Y. Qi, Y. Li, Y. Zhang, Y. Zhang, Y. Adi, Y. Nam, Yu, Wang, Y. Zhao, Y. Hao, Y. Qian, Y. Li, Y. He, Z. Rait, Z. DeVito, Z. Rosnbrick, Z. Wen, Z. Yang, Z. Zhao, and Z. Ma, "The llama 3 herd of models," 2024. [Online]. Available: https://arxiv.org/abs/2407.21783

[22] Y. Gu, A. Khadem, S. Umesh, N. Liang, X. Servot, O. Mutlu, R. Iyer, and R. Das, "Pim is all you need: A cxl-enabled gpu-free system for large language model inference," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. ACM, Mar. 2025, pp. 862–881. [Online]. Available: http://dx.doi.org/10.1145/3676641.3716267

[23] A. Gujarati, R. Karimi, S. Alzayat, W. Hao, A. Kaufmann, Y. Vigfusson, and J. Mace, "Serving DNNs like clockwork: Performance predictability from the bottom up," in *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association, Nov. 2020, pp. 443–462. [Online]. Available: https://www.usenix.org/conference/osdi20/presentation/gujarati

[24] J. Gómez-Luna, I. E. Hajj, I. Fernandez, C. Giannoula, G. F. Oliveira, and O. Mutlu, "Benchmarking a new paradigm: Experimental analysis and characterization of a real processing-in-memory system," *IEEE Access*, vol. 10, pp. 52 565–52 608, 2022.

[25] J. He and J. Zhai, "Fastdecode: High-throughput gpu-efficient llm serving using heterogeneous pipelines," 2024. [Online]. Available: https://arxiv.org/abs/2403.11421

[26] M. He, C. Song, I. Kim, C. Jeong, S. Kim, I. Park, M. Thottethodi, and T. N. Vijaykumar, "Newton: A dram-maker's accelerator-in-memory (aim) architecture for machine learning," in *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2020, pp. 372–385.

[27] Y. He, H. Mao, C. Giannoula, M. Sadrosadati, J. Gómez-Luna, H. Li, X. Li, Y. Wang, and O. Mutlu, "Papi: Exploiting dynamic parallelism in large language model decoding with a processing-in-memory-enabled computing system," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ser. ASPLOS '25. New York, NY, USA: Association for Computing Machinery, 2025, pp. 766–782. [Online]. Available: https://doi.org/10.1145/3676641.3716009

[28] G. Heo, S. Lee, J. Cho, H. Choi, S. Lee, H. Ham, G. Kim, D. Mahajan, and J. Park, "Neupims: Npu-pim heterogeneous acceleration for batched llm inferencing," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, ser. ASPLOS '24. New York, NY, USA: Association for Computing Machinery, 2024, pp. 722–737. [Online]. Available: https://doi.org/10.1145/3620666.3651380

[29] J. Huang and K. C.-C. Chang, "Towards reasoning in large language models: A survey," 2023. [Online]. Available: https://arxiv.org/abs/2212.10403

[30] X. Jiang, Y. Zhou, S. Cao, I. Stoica, and M. Yu, "Neo: Saving gpu memory crisis with cpu offloading for online llm inference," 2024. [Online]. Available: https://arxiv.org/abs/2411.01142

[31] N. P. Jouppi, A. B. Kahng, N. Muralimanohar, and V. Srinivas, "Cacti-io: Cacti with off-chip power-area-timing models," in *2012 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2012, pp. 294–301.

[32] A. K. Kakolyris, D. Masouros, P. Vavaroutsos, S. Xydis, and D. Soudris, "throttll'em: Predictive gpu throttling for energy efficient llm inference serving," in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2025, pp. 1363–1378.

[33] K. S. Kalyan, "A survey of gpt-3 family large language models including chatgpt and gpt-4," 2023. [Online]. Available: https://arxiv.org/abs/2310.12321

[34] L. Ke, U. Gupta, B. Y. Cho, D. Brooks, V. Chandra, U. Diril, A. Firoozshahian, K. Hazelwood, B. Jia, H.-H. S. Lee, M. Li, B. Maher, D. Mudigere, M. Naumov, M. Schatz, M. Smelyanskiy, X. Wang, B. Reagen, C.-J. Wu, M. Hempstead, and X. Zhang, "Recnmp: Accelerating personalized recommendation with near-memory processing," in *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, 2020, pp. 790–803.

[35] L. Ke, X. Zhang, J. So, J.-G. Lee, S.-H. Kang, S. Lee, S. Han, Y. Cho, J. H. Kim, Y. Kwon, K. Kim, J. Jung, I. Yun, S. J. Park, H. Park, J. Song, J. Cho, K. Sohn, N. S. Kim, and H.-H. S. Lee, "Near-memory processing in action: Accelerating personalized recommendation with axdimm," *IEEE Micro*, vol. 42, no. 1, pp. 116–127, 2022.

[36] G. Kim, J. Kim, N. Kim, W. Shin, J. Won, H. Joo, H. Choi, B. An, G. Shin, D. Yun, J. Kim, C. Kim, I. Kim, J. Park, Y. Song, B. Yang, H. Lee, S. Park, W. Lee, S. Kim, Y. Park, Y. Jung, G.-H. Park, and E. Lim, "Sk hynix ai-specific computing memory solution: From aim device to heterogeneous aimx-xpu system for comprehensive llm inference," in *2024 IEEE Hot Chips 36 Symposium (HCS)*, 2024, pp. 1–26.

[37] H. Kim, H. Park, T. Kim, K. Cho, E. Lee, S. Ryu, H.-J. Lee, K. Choi, and J. Lee, "Gradpim: A practical processing-in-dram architecture for gradient descent," in *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2021, pp. 249–262.

[38] J. H. Kim, S.-h. Kang, S. Lee, H. Kim, W. Song, Y. Ro, S. Lee, D. Wang, H. Shin, B. Phuah, J. Choi, J. So, Y. Cho, J. Song, J. Choi, J. Cho, K. Sohn, Y. Sohn, K. Park, and N. S. Kim, "Aquabolt-xl: Samsung hbm2-pim with

in-memory processing for ml accelerators and beyond," in *2021 IEEE Hot Chips 33 Symposium (HCS)*, 2021, pp. 1–26.

[39] J. H. Kim, S.-h. Kang, S. Lee, H. Kim, W. Song, Y. Ro, S. Lee, D. Wang, H. Shin, B. Phuah, J. Choi, J. So, Y. Cho, J. Song, J. Choi, J. Cho, K. Sohn, Y. Sohn, K. Park, and N. S. Kim, "Aquabolt-xl: Samsung hbm2-pim with in-memory processing for ml accelerators and beyond," in *2021 IEEE Hot Chips 33 Symposium (HCS)*, 2021, pp. 1–26.

[40] H. Kwon, K. Koo, J. Kim, W. Lee, M. Lee, H. Lee, Y. Jung, J. Park, Y. Song, B. Yang, H. Choi, G. Kim, J. Won, W. Shin, C. Kim, G. Shin, Y. Kwon, I. Kim, E. Lim, J. Kim, and J. Choi, "Lol-pim: Long-context llm decoding with scalable dram-pim system," 2025. [Online]. Available: https://arxiv.org/abs/2412.20166

[41] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with pagedattention," in *Proceedings of the 29th Symposium on Operating Systems Principles*, ser. SOSP '23. New York, NY, USA: Association for Computing Machinery, 2023, pp. 611–626. [Online]. Available: https://doi.org/10.1145/3600006.3613165

[42] Y. Kwon, G. Kim, N. Kim, W. Shin, J. Won, H. Joo, H. Choi, B. An, G. Shin, D. Yun, J. Kim, C. Kim, I. Kim, J. Park, C. Park, Y. Song, B. Yang, H. Lee, S. Park, W. Lee, S. Lee, K. Kim, D. Kwon, C. Jeong, J. Kim, E. Lim, and J. Chun, "Memory-centric computing with sk hynix's domain-specific memory," in *2023 IEEE Hot Chips 35 Symposium (HCS)*, 2023, pp. 1–26.

[43] Y. Kwon, K. Vladimir, N. Kim, W. Shin, J. Won, M. Lee, H. Joo, H. Choi, G. Kim, B. An, J. Kim, J. Lee, I. Kim, J. Park, C. Park, Y. Song, B. Yang, H. Lee, S. Kim, D. Kwon, S. Lee, K. Kim, S. Oh, J. Park, G. Hong, D. Ka, K. Hwang, J. Park, K. Kang, J. Kim, J. Jeon, M. Lee, M. Shin, M. Shin, J. Cha, C. Jung, K. Chang, C. Jeong, E. Lim, I. Park, J. Chun, and S. Hynix, "System architecture and software stack for gddr6-aim," in *2022 IEEE Hot Chips 34 Symposium (HCS)*, 2022, pp. 1–25.

[44] D. Lee, J. So, M. AHN, J.-G. Lee, J. Kim, J. Cho, R. Oliver, V. C. Thummala, R. s. JV, S. S. Upadhya, M. I. Khan, and J. H. Kim, "Improving in-memory database operations with acceleration dimm (axdimm)," in *Proceedings of the 18th International Workshop on Data Management on New Hardware*, ser. DaMoN '22. New York, NY, USA: Association for Computing Machinery, 2022. [Online]. Available: https://doi.org/10.1145/3533737.3535093

[45] D. Lee, B. Hyun, T. Kim, and M. Rhu, "Pim-mmu: A memory management unit for accelerating data transfers in commercial pim systems," in *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2024, pp. 627–642.

[46] H. Lee, D. Baek, J. Son, J. Choi, K. Moon, and M. Jang, "Paise: Pim-accelerated inference scheduling engine for transformer-based llm," in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2025, pp. 1707–1719.

[47] H. Lee, G. Kim, D. Yun, I. Kim, Y. Kwon, and E. Lim, "Cost-effective llm accelerator using processing in memory technology," in *2024 IEEE Symposium on VLSI Technology and Circuits (VLSI Technology and Circuits)*, 2024, pp. 1–2.

[48] S. Lee, K. Kim, S. Oh, J. Park, G. Hong, D. Ka, K. Hwang, J. Park, K. Kang, J. Kim, J. Jeon, N. Kim, Y. Kwon, K. Vladimir, W. Shin, J. Won, M. Lee, H. Joo, H. Choi, J. Lee, D. Ko, Y. Jun, K. Cho, I. Kim, C. Song, C. Jeong, D. Kwon, J. Jang, I. Park, J. Chun, and J. Cho, "A 1ynm 1.25v 8gb, 16gb/s/pin gddr6-based accelerator-in-memory supporting 1tflops mac operation and various activation functions for deep-learning applications," in *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 65, 2022, pp. 1–3.

[49] S. Lee, S.-h. Kang, J. Lee, H. Kim, E. Lee, S. Seo, H. Yoon, S. Lee, K. Lim, H. Shin, J. Kim, O. Seongil, A. Iyer, D. Wang, K. Sohn, and N. S. Kim, "Hardware architecture and software stack for pim based on commercial dram technology : Industrial product," in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*, 2021, pp. 43–56.

[50] C. Li, Y. Yin, X. Wu, J. Zhu, Z. Gao, D. Niu, Q. Wu, X. Si, Y. Xie, C. Zhang, and G. Sun, "H2-llm: Hardware-dataflow co-exploration for heterogeneous hybrid-bonding-based low-batch llm inference," in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, ser. ISCA '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 194–210. [Online]. Available: https://doi.org/10.1145/3695053.3731008

[51] S. Li, Z. Yang, D. Reddy, A. Srivastava, and B. Jacob, "Dramsim3: A cycle-accurate, thermal-capable dram simulator," *IEEE Comput. Archit. Lett.*, vol. 19, no. 2, pp. 106–109, Jul. 2020. [Online]. Available: https://doi.org/10.1109/LCA.2020.2973991

[52] L. Liu, S. Zhao, B. Li, H. Ren, Z. Xu, M. Wang, X. Li, Y. Han, and Y. Wang, "Make llm inference affordable to everyone: Augmenting gpu memory with ndp-dimm," 2025. [Online]. Available: https://arxiv.org/abs/2502.16963

[53] Q. Liu, Y. Yang, D. Du, Y. Xia, P. Zhang, J. Feng, J. R. Larus, and H. Chen, "Harmonizing efficiency and practicability: Optimizing resource utilization in serverless computing with jiagu," in *2024 USENIX Annual Technical Conference (USENIX ATC 24)*. Santa Clara, CA: USENIX Association, Jul. 2024, pp. 1–17. [Online]. Available: https://www.usenix.org/conference/atc24/presentation/liu-qingyuan

[54] D. Lo, L. Cheng, R. Govindaraju, P. Ranganathan, and C. Kozyrakis, "Heracles: improving resource efficiency at scale," vol. 43, no. 3S. New York, NY, USA: Association for Computing Machinery, Jun. 2015, p. 450–462. [Online]. Available: https://doi.org/10.1145/2872887.2749475

[55] "DRAM power calculator," [Online], Micron Technology, 2017, avalable: https://www.micron.com/sales-support/design-tools/dram-power-calculator?srsltid=AfmBOoq5flElzP8dXZW59LhcNk6lRj8mlJb2x0MlPBcc8fXQlzia8Xwu.

[56] "Technical Note Calculating Memory Power for DDR4 SDRAM Introduction," [Online], Micron Technology, 2017, avalable: https://www.mouser.com/pdfDocs/tn4007_ddr4_power_calculation.pdf?srsltid=AfmBOor9Axc_mw_rsqGmkNM8EWV-kw7xYUOnTICSjstofcX5TH2b6vJB.

[57] "NVIDIA DGX A100," [Online], NVIDIA, 2023, avalable: https://resources.nvidia.com/enus-dgx-systems/dgx-ai.

[58] "NVIDIA Blackwell. The engine of the new industrial revolution." [Online], NVIDIA, 2026, avalable: https://www.primeline-solutions.com/media/categories/server/nach-gpu/nvidia-hgx-h200/nvidia-blackwell-b200-datasheet.pdf.

[59] M. O'Connor, N. Chatterjee, D. Lee, J. Wilson, A. Agrawal, S. W. Keckler, and W. J. Dally, "Fine-grained dram: Energy-efficient dram for extreme bandwidth systems," in *2017 50th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2017, pp. 41–54.

[60] X. Pan, E. Li, Q. Li, S. Liang, Y. Shan, K. Zhou, Y. Luo, X. Wang, and J. Zhang, "Instattention: In-storage attention offloading for cost-effective long-context llm inference," in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, March 2025, pp. 1510–1525.

[61] J. Park, J. Choi, K. Kyung, M. J. Kim, Y. Kwon, N. S. Kim, and J. H. Ahn, "Attacc! unleashing the power of pim for batched transformer-based generative model inference," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ser. ASPLOS '24. New York, NY, USA: Association for Computing Machinery, 2024, pp. 103–119. [Online]. Available: https://doi.org/10.1145/3620665.3640422

[62] J. Park, B. Kim, S. Yun, E. Lee, M. Rhu, and J. H. Ahn, "Trim: Enhancing processor-memory interfaces with scalable tensor reduction in memory," in *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO '21. New York, NY, USA: Association for Computing Machinery, 2021, pp. 268–281. [Online]. Available: https://doi.org/10.1145/3466752.3480080

[63] S.-S. Park, K. Kim, J. So, J. Jung, J. Lee, K. Woo, N. Kim, Y. Lee, H. Kim, Y. Kwon, J. Kim, J. Lee, Y. Cho, Y. Tai, J. Cho, H. Song, J. H. Ahn, and N. S. Kim, "An lpddr-based cxl-pnm platform for tco-efficient inference of transformer-based large language models," in *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2024, pp. 970–982.

[64] "PCI Express Specifications," [Online], PCI-SIG, 2025, avalable: https://pcisig.com/specifications.

[65] A. Plaat, A. Wong, S. Verberne, J. Broekens, N. van Stein, and T. Back, "Reasoning with large language models, a survey," 2024. [Online]. Available: https://arxiv.org/abs/2407.11511

[66] M. Rhee, J. Sim, T. Ahn, S. Lee, D. Yoon, E. Kim, K. Park, Y. Joo, and H. Kim, "Hpu: High-bandwidth processing unit for scalable, cost-effective llm inference via gpu co-processing," *arXiv preprint arXiv:2504.16112*, 2025.

[67] M. Seo, X. T. Nguyen, S. J. Hwang, Y. Kwon, G. Kim, C. Park, I. Kim, J. Park, J. Kim, W. Shin, J. Won, H. Choi, K. Kim, D. Kwon, C. Jeong, S. Lee, Y. Choi, W. Byun, S. Baek, H.-J. Lee, and J. Kim, "Ianus: Integrated accelerator based on npu-pim unified memory system," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating*

Systems, Volume 3, ser. ASPLOS '24. New York, NY, USA: Association for Computing Machinery, 2024, pp. 545–560. [Online]. Available: https://doi.org/10.1145/3620666.3651324

[68] S. H. Seo, J. Kim, D. Lee, S. Yoo, S. Moon, Y. Park, and J. W. Lee, "Facil: Flexible dram address mapping for soc-pim cooperative on-device llm inference," in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2025, pp. 1720–1733.

[69] Y. Sheng, L. Zheng, B. Yuan, Z. Li, M. Ryabinin, B. Chen, P. Liang, C. Ré, I. Stoica, and C. Zhang, "Flexgen: high-throughput generative inference of large language models with a single gpu," in *Proceedings of the 40th International Conference on Machine Learning*, ser. ICML'23. JMLR.org, 2023.

[70] J. Stojkovic, C. Zhang, I. Goiri, J. Torrellas, and E. Choukse, "Dynamollm: Designing llm inference clusters for performance and energy efficiency," in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2025, pp. 1348–1362.

[71] S. Team, "Step3: Cost-effective multimodal intelligence." [Online]. Available: https://stepfun.ai/research/step3

[72] S. Team, "Step-3 is large yet affordable: Model-system co-design for cost-effective decoding," 2025. [Online]. Available: https://arxiv.org/abs/2507.19427

[73] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Proceedings of the 31st International Conference on Neural Information Processing Systems*, ser. NIPS'17. Red Hook, NY, USA: Curran Associates Inc., 2017, pp. 6000–6010.

[74] Y. Wei, Y. C. Huang, H. Tang, N. Sankaran, I. Chadha, D. Dai, O. Oluwole, V. Balan, and E. Lee, "9.3 nvlink-c2c: A coherent off package chip-to-chip interconnect with 40gbps/pin single-ended signaling," in *2023 IEEE International Solid-State Circuits Conference (ISSCC)*, 2023, pp. 160–162.

[75] Wikipedia, "Liebig's law of the minimum," 2025. [Online]. Available: https://en.wikipedia.org/wiki/Liebig%27s_law_of_the_minimum

[76] S. Williams, A. Waterman, and D. Patterson, "Roofline: an insightful visual performance model for multicore architectures," *Commun. ACM*, vol. 52, no. 4, p. 65–76, Apr. 2009. [Online]. Available: https://doi.org/10.1145/1498765.1498785

[77] Y. Yang, L. Zhao, Y. Li, H. Zhang, J. Li, M. Zhao, X. Chen, and K. Li, "Infless: a native serverless system for low-latency, high-throughput inference," in *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 768–781. [Online]. Available: https://doi.org/10.1145/3503222.3507709

[78] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, "Orca: A distributed serving system for Transformer-Based generative models," in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. Carlsbad, CA: USENIX Association, Jul. 2022, pp. 521–538. [Online]. Available: https://www.usenix.org/conference/osdi22/presentation/yu

[79] S. Yun, K. Kyung, J. Cho, J. Choi, J. Kim, B. Kim, S. Lee, K. Sohn, and J. H. Ahn, "Duplex: A device for large language models with mixture of experts, grouped query attention, and continuous batching," in *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2024, pp. 1429–1443.

[80] L. Zhao, Y. Yang, Y. Li, X. Zhou, and K. Li, "Understanding, predicting and scheduling serverless workloads under partial interference," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, ser. SC '21. New York, NY, USA: Association for Computing Machinery, 2021. [Online]. Available: https://doi.org/10.1145/3458817.3476215

[81] T. Zhong, Z. Liu, Y. Pan, Y. Zhang, Y. Zhou, S. Liang, Z. Wu, Y. Lyu, P. Shu, X. Yu, C. Cao, H. Jiang, H. Chen, Y. Li, J. Chen, H. Hu, Y. Liu, H. Zhao, S. Xu, H. Dai, L. Zhao, R. Zhang, W. Zhao, Z. Yang, J. Chen, P. Wang, W. Ruan, H. Wang, H. Zhao, J. Zhang, Y. Ren, S. Qin, T. Chen, J. Li, A. H. Zidan, A. Jahin, M. Chen, S. Xia, J. Holmes, Y. Zhuang, J. Wang, B. Xu, W. Xia, J. Yu, K. Tang, Y. Yang, B. Sun, T. Yang, G. Lu, X. Wang, L. Chai, H. Li, J. Lu, L. Sun, X. Zhang, B. Ge, X. Hu, L. Zhang, H. Zhou, L. Zhang, S. Zhang, N. Liu, B. Jiang, L. Kong, Z. Xiang, Y. Ren, J. Liu, X. Jiang, Y. Bao, W. Zhang, X. Li, G. Li, W. Liu, D. Shen, A. Sikora, X. Zhai, D. Zhu, and T. Liu, "Evaluation of openai o1: Opportunities and challenges of agi," 2024. [Online]. Available: https://arxiv.org/abs/2409.18486

[82] Z. Zhou, X. Ning, K. Hong, T. Fu, J. Xu, S. Li, Y. Lou, L. Wang, Z. Yuan, X. Li, S. Yan, G. Dai, X.-P. Zhang, Y. Dong, and Y. Wang, "A survey on efficient inference for large language models," 2024. [Online]. Available: https://arxiv.org/abs/2404.14294

[83] R. Zhu, Z. Jiang, C. Jin, P. Wu, C. A. Stuardo, D. Wang, X. Zhang, H. Zhou, H. Wei, Y. Cheng, J. Xiao, X. Zhang, L. Liu, H. Lin, L.-W. Chang, J. Ye, X. Yu, X. Liu, X. Jin, and X. Liu, "Megascale-infer: Efficient mixture-of-experts model serving with disaggregated expert parallelism," in *Proceedings of the ACM SIGCOMM 2025 Conference*, ser. SIGCOMM '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 592–608. [Online]. Available: https://doi.org/10.1145/3718958.3750506